Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔷 SQL Server DBA Performance Tuning & Optimization – 100 Real-Time Q&A

1. Performance Troubleshooting Basics

1. **Q:** A SQL Server is running slow suddenly. What's the first thing you check?
   A: Start with sp_whoisactive / Activity Monitor to see active sessions, blocking chains, high CPU queries, or waits. Then check PerfMon counters (CPU > 80%, Page Life Expectancy < 300, Disk Latency > 20ms).
2. **Q:** How do you differentiate between SQL Server and OS-level performance issues?
   A: Compare SQL Server DMVs (sys.dm_exec_requests, sys.dm_os_wait_stats) with OS tools (Task Manager, PerfMon). If SQL CPU usage matches system CPU usage → SQL is consuming; otherwise, other processes are.
3. **Q:** A system is running with high CPU usage. Which DMV helps identify problematic queries?
   A: Use sys.dm_exec_query_stats with CROSS APPLY sys.dm_exec_sql_text() to find top CPU-consuming queries.
4. **Q:** What is Page Life Expectancy (PLE), and what does a low value mean?
   A: PLE is how long a data page stays in memory (Buffer Pool). A consistently low PLE (<300 seconds) indicates memory pressure.
5. **Q:** How do you check if a query is CPU-bound or I/O-bound?
   A: Use SET STATISTICS IO, TIME ON. If CPU time is high, it's CPU-bound; if Logical/Physical Reads are high, it's I/O-bound.

2. Indexes & Execution Plans

6. **Q:** How do missing indexes affect performance?
   A: Queries perform table scans instead of index seeks → higher I/O. Use sys.dm_db_missing_index_details.
7. **Q:** What is an execution plan, and why is it important?
   A: It shows how SQL optimizes a query (scan vs. seek, join type). Helps spot inefficiencies like key lookups, sorts, scans.
8. **Q:** How do you eliminate "Key Lookup" issues?
   A: Add a **covering index** that includes the columns being selected.
9. **Q:** How do you detect unused indexes?
   A: Use sys.dm_db_index_usage_stats to find indexes never used but consuming space during writes.
10. **Q:** How do you reduce fragmentation in indexes?
    A: Rebuild indexes if fragmentation >30%, reorganize if 10–30%. Use sys.dm_db_index_physical_stats.

3. Query Optimization

11. **Q:** A query runs fine with small data but is slow with large data. Why?
    A: Poor indexing, parameter sniffing, statistics not updated, or inefficient query design.
12. **Q:** What is parameter sniffing?
    A: SQL caches a plan based on first parameter values. Subsequent executions with different parameters may be inefficient.
13. **Q:** How do you fix parameter sniffing issues?
    A: Options: OPTION (RECOMPILE), use local variables, or optimize indexes/statistics.
14. **Q:** Difference between JOIN and EXISTS for filtering?
    A: EXISTS is faster for checking existence (stops on first match). JOIN may create duplicates.
15. **Q:** Why is SELECT * bad for performance?
    A: Fetches unnecessary columns, prevents index covering, increases I/O & network load.

4. Wait Statistics & Blocking

16. **Q:** How do you identify blocking in SQL Server?
    A: Use sp_who2, sp_whoisactive, or DMV sys.dm_exec_requests with blocking session ID.
17. **Q:** What are Wait Statistics in SQL Server?
    A: They show where SQL spends most of its time (CPU waits, I/O waits, locking waits).

18. **Q:** Which wait types indicate disk bottlenecks?
    A: PAGEIOLATCH_*, WRITELOG.
19. **Q:** Which wait types indicate CPU pressure?
    A: SOS_SCHEDULER_YIELD, CXPACKET (parallelism).
20. **Q:** How do you resolve blocking?
    A: Kill blocking session (last resort), add proper indexes, reduce transaction scope, use NOLOCK carefully.

5. Statistics & Cardinality

21. **Q:** Why are statistics important for performance?
    A: SQL uses stats to estimate row counts → affects plan choice. Outdated stats lead to wrong plans.
22. **Q:** How do you update statistics?
    A: UPDATE STATISTICS TableName or sp_updatestats.
23. **Q:** When should AUTO_UPDATE_STATISTICS_ASYNC be used?
    A: For workloads needing fast response → prevents query blocking due to synchronous stats updates.
24. **Q:** How do filtered statistics improve performance?
    A: Target skewed data distributions → better row estimation.
25. **Q:** What is the difference between histograms and density vectors in statistics?
    A: Histograms store value distribution of a column, density vectors store uniqueness for joins.

6. Memory & TempDB

26. **Q:** How do you check memory usage by SQL Server?
    A: sys.dm_os_process_memory and sys.dm_os_memory_clerks.
27. **Q:** What is a memory grant, and why does it cause performance issues?
    A: Queries request memory for sorting/joining. If underestimated → spills to TempDB.
28. **Q:** How do you detect TempDB contention?
    A: High PAGELATCH waits. Use multiple TempDB data files (1 per core up to 8).
29. **Q:** How do you reduce TempDB usage?
    A: Avoid temp tables for trivial queries, use indexed temp tables, optimize hash joins.
30. **Q:** How do MAXDOP settings affect performance?
    A: Controls parallelism. Too high → excessive CPU usage; too low → slow performance.

7. I/O & Disk Optimization

31. **Q:** How do you detect slow I/O in SQL Server?
    A: Check sys.dm_io_virtual_file_stats for latency >20ms.
32. **Q:** How do you improve log file performance?
    A: Place log files on separate disks, enable instant file initialization (for data files only), pre-size logs.
33. **Q:** How do you detect “autogrowth” issues?
    A: Frequent growth events in Error Log. Fix: Pre-size DB and set proper growth.
34. **Q:** Why are multiple data files used in large databases?
    A: To reduce allocation contention (especially in TempDB).
35. **Q:** What RAID levels are best for SQL Server?
    A: RAID 10 (best balance of speed + redundancy), RAID 5 (cheap but slower for writes).

8. Performance Monitoring Tools

36. **Q:** Which DMVs are most useful for performance tuning?
    A: sys.dm_exec_requests, sys.dm_exec_query_stats, sys.dm_os_wait_stats, sys.dm_io_virtual_file_stats.
37. **Q:** What is the difference between Profiler and Extended Events?
    A: Profiler = legacy, more overhead. Extended Events = lightweight, modern alternative.

38. **Q:** How do you capture long-running queries?
    A: Extended Events, SQL Trace, Query Store.
39. **Q:** What is Query Store?
    A: Feature to capture execution history & force good plans.
40. **Q:** How do you baseline SQL Server performance?
    A: Regularly capture wait stats, PerfMon counters, IO latency, query performance.

9. Optimization Scenarios

41. **Q:** A stored procedure runs fast in SSMS but slow in application. Why?
    A: Different SET options, parameter sniffing, different execution plans.
42. **Q:** Why do queries sometimes run slower after an index rebuild?
    A: Plan changes due to updated stats, fragmentation reset.
43. **Q:** Why does "TOP 1 ORDER BY" sometimes cause performance issues?
    A: Requires sorting entire dataset. Fix: Use proper index on ORDER BY column.
44. **Q:** How do you optimize bulk inserts?
    A: Use TABLOCK, disable indexes during load, batch inserts.
45. **Q:** How do you tune queries with multiple joins?
    A: Ensure indexes on join keys, eliminate unnecessary columns, check join order in execution plan.

10. Advanced Optimization

46. **Q:** What is forced parameterization?
    A: SQL reuses plans by treating literals as parameters. Helps reduce plan cache bloat.
47. **Q:** What are hints, and when should you use them?
    A: Hints force specific behaviors (e.g., NOLOCK, OPTIMIZE FOR). Use only when optimizer fails.
48. **Q:** What is Adaptive Query Processing?
    A: SQL Server 2017+ feature to adjust plans dynamically based on runtime feedback.
49. **Q:** What is Batch Mode processing?
    A: Columnstore optimization for analytical queries → processes rows in batches, faster than row mode.
50. **Q:** What is In-Memory OLTP (Hekaton), and how does it improve performance?
    A: Memory-optimized tables eliminate locking/latching → faster transactions.

11. Query Store & Plan Management

51. **Q:** What is the Query Store, and why is it useful?
    A: It captures query history, execution plans, and performance stats. Useful for spotting plan regressions and forcing stable plans.
52. **Q:** How do you force a good plan using Query Store?
    A: ALTER QUERY in Query Store → "Force Plan" option. Prevents SQL from switching to worse plans.
53. **Q:** What happens if the forced plan becomes invalid?
    A: SQL automatically unforces it and reverts to normal optimization.
54. **Q:** How do you identify regressed queries in Query Store?
    A: Use built-in reports → *Top Resource Consuming Queries* or filter by regressed queries.
55. **Q:** How does Query Store differ from plan cache?
    A: Plan cache is volatile (cleared on restart); Query Store is persistent.

12. Extended Events & Monitoring

56. **Q:** What are Extended Events?
    A: Lightweight monitoring framework for capturing performance events (e.g., long queries, deadlocks).

57. **Q:** How do you capture deadlocks with Extended Events?
    A: Create a session for xml_deadlock_report.
58. **Q:** Why are Extended Events better than SQL Profiler?
    A: Less overhead, asynchronous, richer events.
59. **Q:** How do you capture slow queries with Extended Events?
    A: Create a session for rpc_completed or sql_batch_completed with duration filter.
60. **Q:** Can Extended Events be used in production?
    A: Yes, recommended over Profiler for real-time monitoring.

13. Deadlocks & Concurrency

61. **Q:** How do you detect deadlocks?
    A: SQL Error Log (1222), Extended Events, Profiler, or sys.dm_tran_locks.
62. **Q:** How do you prevent deadlocks?
    A: Access objects in the same order, keep transactions short, use proper indexes.
63. **Q:** What is the deadlock victim?
    A: SQL automatically chooses one transaction to rollback (least costly) when deadlock occurs.
64. **Q:** How do you simulate a deadlock for testing?
    A: Run two transactions accessing the same resources in reverse order.
65. **Q:** What is optimistic vs. pessimistic concurrency?
    A: Optimistic assumes low conflicts (uses row versions); pessimistic locks resources to avoid conflicts.

14. Parallelism & MAXDOP

66. **Q:** What is MAXDOP in SQL Server?
    A: Maximum Degree of Parallelism → controls CPU cores used for queries.
67. **Q:** What causes CXPACKET waits?
    A: Inefficient parallel plans → imbalance in parallel threads.
68. **Q:** How do you fix CXPACKET waits?
    A: Adjust MAXDOP, update statistics, rewrite queries.
69. **Q:** How do you set MAXDOP at query level?
    A: OPTION (MAXDOP 1) in query hint.
70. **Q:** What is the best practice for MAXDOP setting?
    A: Typically # of logical CPUs per NUMA node (up to 8).

15. Indexing Advanced

71. **Q:** What are filtered indexes?
    A: Index with a WHERE clause → reduces size, improves performance for skewed data.
72. **Q:** When should you use columnstore indexes?
    A: For OLAP/analytical queries (large scans, aggregations).
73. **Q:** What is the drawback of columnstore indexes?
    A: Slower OLTP operations (insert, update, delete).
74. **Q:** What are included columns in indexes?
    A: Non-key columns stored at leaf level → reduces key lookups.
75. **Q:** How do indexed views improve performance?
    A: Store pre-aggregated results physically → reduces computation.

16. Always On & High Availability Performance

76. **Q:** How do synchronous and asynchronous AGs affect performance?
    A: Synchronous → adds commit latency; Asynchronous → faster but risk of data loss.

77. **Q:** What is the impact of readable secondaries?
    A: They add CPU/I/O load due to redo thread and tempdb usage.
78. **Q:** How do you reduce AG failover time?
    A: Tune network, pre-size logs, optimize redo queue.
79. **Q:** What is redo queue in AGs?
    A: Unapplied log records on secondary; large queues cause lag.
80. **Q:** How do you monitor AG performance?
    A: DMV sys.dm_hadr_database_replica_states (redo/ send queue size).

17. Replication & Performance

81. **Q:** What replication type impacts performance most?
    A: Merge replication (conflict detection overhead).
82. **Q:** How do you optimize transactional replication?
    A: Use indexed PKs, filter articles, tune log reader agent.
83. **Q:** What slows down replication?
    A: Large transactions, high latency network, unindexed columns.
84. **Q:** How do you monitor replication latency?
    A: sp_replmonitorsubscriptionpendingcmds.
85. **Q:** How do you troubleshoot slow replication agent?
    A: Check agent profile, batch size, max concurrency.

18. TempDB & Contention

86. **Q:** What is PAGELATCH contention in TempDB?
    A: Multiple threads fighting for same page → delays.
87. **Q:** How do you fix TempDB contention?
    A: Use multiple data files (equal size), trace flags 1117 & 1118 (pre-SQL 2016).
88. **Q:** Why is TempDB critical for performance?
    A: Used for sorts, joins, versioning, temp tables.
89. **Q:** What causes TempDB spills?
    A: Insufficient memory grant → sorts/hash joins spill to TempDB.
90. **Q:** How do you monitor TempDB usage?
    A: DMV sys.dm_db_task_space_usage.

19. Backup & Restore Performance

91. **Q:** How do you optimize backup speed?
    A: Use striped backups, compression, backup to disk instead of tape.
92. **Q:** Does backup compression impact performance?
    A: Saves I/O, but adds CPU overhead.
93. **Q:** How do you test restore speed?
    A: RESTORE VERIFYONLY for integrity, but actual restore for speed.
94. **Q:** How do instant file initialization help restores?
    A: Skips zeroing data files → faster restores.
95. **Q:** How do you monitor backup performance?
    A: DMV sys.dm_io_virtual_file_stats, MSDB backup history.

20. Cloud & Miscellaneous

96. **Q:** How does Azure SQL performance tuning differ from on-prem?
    A: Limited server access; focus on DTU/vCore monitoring, Query Store, automatic tuning.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

97. **Q:** What is automatic tuning in SQL Azure?
    A: SQL auto-creates/drops indexes and forces plans.
98. **Q:** How do you handle noisy neighbor problems in cloud SQL?
    A: Scale up service tier, enable resource governance.
99. **Q:** What are Resource Governor best practices?
    A: Use for multi-tenant workloads to limit CPU/IO.
100. **Q:** How do you approach end-to-end performance tuning?
    A: Baseline → Identify bottleneck (CPU, IO, Memory, Network) → Tune queries/indexes → Monitor waits → Test fixes.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔷 SQL Server DBA – Top 50 Performance Troubleshooting & Resolution FAQ (With Detailed Answers)

1. General Troubleshooting

1. **Q:** SQL Server suddenly becomes slow—what's the first step?
   A: Check system health (CPU, memory, I/O) using Task Manager & PerfMon. Then in SQL, run sp_whoisactive or sys.dm_exec_requests to find blocking, long queries, or high waits. Always differentiate SQL issues vs. OS/hardware issues first.
2. **Q:** How do you identify if slowness is due to SQL or external factors (like network)?
   A: Compare SQL DMV performance (CPU waits, I/O stalls) vs. OS counters. If SQL metrics are clean but app still slow, investigate network latency, firewall, or app tier.
3. **Q:** A query is slow in production but fast in test. What's your troubleshooting path?
   A: Compare execution plans, statistics freshness, index differences, server resources, and parameter sniffing. Often test DB is smaller, so scans appear "fast" there.
4. **Q:** How do you troubleshoot sudden CPU spikes?
   A: Run DMV:
5. SELECT TOP 10 total_worker_time/execution_count AS avg_cpu, text
6. FROM sys.dm_exec_query_stats CROSS APPLY sys.dm_exec_sql_text(sql_handle)
7. ORDER BY avg_cpu DESC;

This shows the top CPU consumers.

8. **Q:** What's your approach if users complain of "SQL Server is hanging"?
   A: Check:
   - Blocking sessions (sp_whoisactive)
   - Deadlocks (system_health XE session)
   - Resource waits (sys.dm_os_wait_stats)
   - Error log for hardware/IO errors

2. Blocking & Deadlocks

6. **Q:** How do you detect blocking?
   A: sp_whoisactive → look for blocked by column. DMV sys.dm_exec_requests also shows blocking_session_id.
7. **Q:** What's the difference between blocking and deadlock?
   A: Blocking = one query waits for another. Deadlock = two queries cyclically wait → SQL kills one as victim.
8. **Q:** How do you capture deadlocks?
   A: Use Extended Events (xml_deadlock_report) or trace flag 1222 to log into SQL Error Log.
9. **Q:** How do you resolve blocking issues permanently?
   A: Tune queries causing long locks, create proper indexes, reduce transaction scope, consider optimistic isolation (RCSI).
10. **Q:** How do you minimize deadlocks?
    A: Access resources in same order, commit transactions quickly, add covering indexes.

3. Wait Statistics

11. **Q:** What are Wait Stats, and why important?
    A: They show where SQL spends time waiting (CPU, I/O, locks). Key for root cause troubleshooting.
12. **Q:** Which wait types indicate I/O problems?
    A: PAGEIOLATCH_*, WRITELOG. They mean slow disk reads/writes.
13. **Q:** Which waits indicate CPU bottleneck?
    A: SOS_SCHEDULER_YIELD, high CXPACKET (parallelism).
14. **Q:** How do you clear wait stats for troubleshooting?
    A: DBCC SQLPERF('sys.dm_os_wait_stats', CLEAR)—then monitor fresh waits.

15. **Q:** What's the difference between LATCH and LOCK waits?
    A: LATCH = internal SQL memory/page structures; LOCK = user transactions/resources.

4. Query Performance Issues

16. **Q:** A query runs fast in SSMS but slow in app—why?
    A: Different SET options, parameter sniffing, network round-trips, or different execution context.
17. **Q:** What is parameter sniffing?
    A: SQL caches plan based on first parameter. Future executions with skewed data may suffer.
18. **Q:** How do you fix parameter sniffing?
    A: Use OPTIMIZE FOR, RECOMPILE, or rewrite proc with local variables.
19. **Q:** Why avoid SELECT *?
    A: Causes table scans, prevents index covering, increases I/O & network traffic.
20. **Q:** A stored procedure runs slow after deployment. What's the checklist?
    A: Recompile, check execution plan, verify stats, compare indexes between environments.

5. Index & Statistics

21. **Q:** How do you detect missing indexes?
    A: DMV sys.dm_db_missing_index_details. Also check execution plans for "Missing Index" warnings.
22. **Q:** How do you find unused indexes?
    A: DMV sys.dm_db_index_usage_stats. Drop those with no reads but heavy writes.
23. **Q:** What's fragmentation, and how do you fix it?
    A: Logical ordering mismatch → slower scans. Fix by ALTER INDEX REORGANIZE/REBUILD.
24. **Q:** How often update statistics?
    A: Auto-update ON by default. But for volatile tables, schedule UPDATE STATISTICS.
25. **Q:** What's a covering index?
    A: Index that includes all queried columns → avoids key lookups.

6. TempDB & Memory

26. **Q:** Why is TempDB a common performance bottleneck?
    A: Used for temp tables, sorts, version store, row versioning. Heavy contention = system slowdown.
27. **Q:** How do you fix TempDB contention?
    A: Add multiple equally sized data files (1 per core up to 8), enable trace flag 1118 (SQL <2016).
28. **Q:** How to detect memory pressure?
    A: Low Page Life Expectancy (PLE < 300), high memory grants pending, frequent IO.
29. **Q:** What is a memory grant?
    A: Memory requested for sorts/joins. If underestimated, query spills to TempDB.
30. **Q:** How do you identify memory-hungry queries?
    A: DMV: sys.dm_exec_query_memory_grants.

7. I/O & Disk Troubleshooting

31. **Q:** How to check disk latency from SQL?
    A: DMV sys.dm_io_virtual_file_stats. Avg Read/Write Latency >20ms = bad.
32. **Q:** What causes WRITELOG waits?
    A: Slow log disk, small log files growing frequently.
33. **Q:** How do you optimize transaction log writes?
    A: Pre-size logs, put logs on dedicated fast disk, use multiple VLFs.
34. **Q:** What's Instant File Initialization?
    A: Allows data files to skip zeroing → faster growth/restores (not for log files).

35. **Q:** How to detect auto-growth performance issues?
    A: Frequent growth events in SQL Error Log. Fix: Pre-size and set proper growth increments.

8. Parallelism & CPU

36. **Q:** What is MAXDOP?
    A: Maximum Degree of Parallelism. Controls # of CPUs per query.
37. **Q:** What causes CXPACKET waits?
    A: Imbalanced parallel plans or wrong MAXDOP.
38. **Q:** How do you tune parallelism?
    A: Set MAXDOP = number of cores per NUMA node (≤8), use OPTION (MAXDOP 1) for OLTP queries.
39. **Q:** Why can disabling parallelism hurt?
    A: Analytical/large queries benefit from parallel execution.
40. **Q:** How do you detect high CPU queries?
    A: DMV sys.dm_exec_query_stats ORDER BY total_worker_time.

9. Monitoring & Tools

41. **Q:** What's better: Profiler or Extended Events?
    A: Extended Events → lightweight, production-safe, richer event capture.
42. **Q:** How do you baseline SQL Server performance?
    A: Capture waits, PerfMon counters, IO latencies, query performance at steady state.
43. **Q:** How do you capture long-running queries?
    A: Use Extended Events or Query Store reports.
44. **Q:** What's the benefit of Query Store?
    A: Captures execution history & plan regressions; allows forcing stable plans.
45. **Q:** How do you monitor real-time active queries?
    A: DMV sys.dm_exec_requests or sp_whoisactive.

10. Real-Time Scenarios

46. **Q:** SQL is slow every morning at 9 AM—how do you debug?
    A: Look for scheduled jobs, index maintenance, backups, or batch workloads at that time.
47. **Q:** A nightly ETL job runs slower over time—what's your approach?
    A: Check for growing table size, fragmentation, outdated stats, increased logging.
48. **Q:** A query is spilling to TempDB—how do you fix it?
    A: Add memory (if feasible), optimize joins, create better indexes, increase worktable efficiency.
49. **Q:** How do you troubleshoot performance issues in AlwaysOn AG?
    A: Check redo/send queue, network latency, synchronous vs. async commit.
50. **Q:** How do you approach performance troubleshooting end-to-end?
    A: Stepwise → Baseline → Identify bottleneck (CPU/IO/Memory/Locks) → Drill into queries/indexes → Apply fixes → Monitor impact.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔷 Top 50 Interview Q&A: Indexes & Execution Plans in SQL Server

Indexes

1. What are indexes in SQL Server and why are they important?

- Indexes are database objects that improve the speed of data retrieval.
- Without indexes, SQL Server scans entire tables (full table scan).
- With indexes, SQL Server can quickly locate data pages using B-Trees.

2. Difference between Clustered and Non-Clustered indexes?

- Clustered Index: Sorts and stores data physically in the table (only one per table).
- Non-Clustered Index: Separate structure with pointers to rows in the clustered index or heap.

3. What is a Heap table?

- A table without a clustered index.
- Data is stored unsorted, retrieval is slower.
- Often leads to forwarding pointers when updated.

4. What are Included Columns in Non-Clustered Indexes?

- Columns added to the index leaf-level for covering queries.
- Improve performance by avoiding key lookups.

5. What is a Covering Index?

- An index that contains all the columns a query needs (either in key or included columns).
- Prevents lookups, improves query speed.

6. What is a Filtered Index?

- An index built with a WHERE clause on a subset of rows.
- Useful for queries filtering on specific values (e.g., active = 1).

7. Difference between Unique Index and Primary Key Index?

- Primary Key: Automatically creates a unique clustered index unless specified otherwise.
- Unique Index: Ensures uniqueness but doesn't enforce primary key constraints.

8. What is Index Fragmentation?

- Internal fragmentation (empty space in pages).
- External fragmentation (logical order of pages mismatched with physical order).
- Fix with **ALTER INDEX REORGANIZE** (low) or **REBUILD** (high).

9. When do you choose REBUILD vs REORGANIZE?

- Reorganize if fragmentation between 5%–30%.
- Rebuild if fragmentation >30%.

10. What is a Fill Factor?

- Percentage of space left empty in index pages during rebuild.
- Helps reduce page splits on future inserts.

11. What are Indexed Views?

- Materialized views that are persisted with clustered index.
- Improve performance but come with overhead on DML.

12. What is the impact of too many indexes?

- Increases DML overhead (insert/update/delete slower).
- More storage used.
- Query optimizer takes longer to choose best index.

13. How do you identify unused indexes?

- Use sys.dm_db_index_usage_stats.
- Look for indexes with low user_seeks/scans and high user_updates.

14. What is a Key Lookup (Bookmark Lookup)?

- When a non-clustered index doesn't have all required columns, SQL Server fetches extra data from clustered index/heap.
- Solution: Covering index or included columns.

15. What are Columnstore Indexes?

- Store data column-wise, highly compressed.
- Best for analytics and large fact tables.
- Improves aggregation and scanning performance.

16. What is the difference between Clustered Columnstore and Non-Clustered Columnstore indexes?

- Clustered Columnstore: Stores entire table in columnar format.
- Non-Clustered Columnstore: Created on top of rowstore table, useful for hybrid OLTP/OLAP.

17. Can we create multiple Clustered Indexes on a table?

- No. Only one clustered index allowed because it defines physical data order.

18. How do you detect Missing Indexes?

- Use sys.dm_db_missing_index_details.
- Execution Plan will also show Missing Index recommendations.

19. What is an XML Index?

- Primary XML Index stores parsed XML data.
- Secondary XML Indexes (PATH, VALUE, PROPERTY) optimize XML queries.

20. What is a Full-text Index?

- Special index used for fast searches on large text columns.
- Supports advanced search features like stemming and synonyms.

Execution Plans

21. What is an Execution Plan?

- A roadmap SQL Server creates to execute a query.
- Shows operations like scans, seeks, joins, sorts, etc.

22. Difference between Estimated and Actual Execution Plan?

- Estimated Plan: Generated before execution (optimizer guess).
- Actual Plan: Generated after execution (includes runtime statistics).

23. How do you generate an Execution Plan?

- In SSMS:
  - Estimated Plan: CTRL + L
  - Actual Plan: CTRL + M

24. What are common operators in Execution Plans?

- Index Seek
- Index Scan
- Table Scan
- Nested Loop Join
- Merge Join
- Hash Match

25. Difference between Index Seek vs Index Scan?

- Seek: Uses index to directly locate rows (efficient).
- Scan: Reads all rows (less efficient).

26. What is a Table Scan?

- Reads every row of the table.
- Happens if no index exists or index is not selective.

27. What is the difference between Nested Loop, Merge Join, and Hash Match?

- Nested Loop: Best for small inputs, index lookups.
- Merge Join: Requires sorted inputs, efficient for large sorted sets.
- Hash Match: Used for large unsorted sets.

28. What is Cardinality Estimation in Execution Plan?

- Process of predicting the number of rows returned at each step.
- Wrong estimation can cause poor plan choice.

29. What are Parameter Sniffing issues in Execution Plans?

- SQL Server caches a plan based on the first parameter value.
- Different parameter values may cause suboptimal plans.
- Fix: Use OPTION (RECOMPILE), Optimize for hint, or plan guides.

30. What is the difference between Actual Rows vs Estimated Rows in Execution Plan?

- Large discrepancies mean cardinality estimation problems, missing stats, or parameter sniffing.

31. How do you detect costly operators in Execution Plans?

- Look for highest **percentage cost** in plan.
- Use Query Store or DMVs for history.

32. What is a Parallel Execution Plan?

- Plan uses multiple threads (operators show "Parallelism").
- Controlled by max degree of parallelism (MAXDOP).

33. What are Execution Plan Warnings?

- Missing Index
- Spill to TempDB (Sort/Hash warnings)
- Implicit conversions

34. What is a "Spill to TempDB" warning?

- Happens when SQL Server runs out of memory for Hash/Sort operations.
- Indicates need for more memory grant or better indexing.

35. How do Statistics affect Execution Plans?

- Optimizer uses statistics to estimate row counts.
- Outdated stats lead to poor plans.

36. How do you update statistics?

- UPDATE STATISTICS or sp_updatestats.
- Use AUTO UPDATE STATISTICS ON for automatic maintenance.

37. What is the difference between SARGable and Non-SARGable queries?

- SARGable queries allow index usage (e.g., WHERE column = 5).
- Non-SARGable (e.g., WHERE YEAR(date) = 2023) forces scans.

38. How do Hints affect Execution Plans?

- FORCESEEK, LOOP JOIN, MAXDOP, OPTIMIZE FOR… can override optimizer choices.
- Useful but should be used cautiously.

39. What is a Plan Cache?

- Stores execution plans for reuse.
- Reduces compilation overhead.

40. What is Plan Reuse vs Plan Recompilation?

- Reuse: SQL Server uses cached plan for similar queries.
- Recompile: SQL Server generates new plan (can be forced with OPTION(RECOMPILE)).

41. How do you identify queries with poor plans?

- Query Store (from SQL Server 2016).
- DMVs like sys.dm_exec_query_stats.

42. What is Forced Parameterization?

- SQL Server replaces literal values with parameters to encourage plan reuse.

43. What is Adaptive Query Processing (SQL Server 2017+)?

- Features like Adaptive Joins, Interleaved Execution, Memory Grant Feedback.
- Helps optimize execution plan at runtime.

44. What is the difference between Execution Plan Cache Bloat and Plan Pollution?

- Cache Bloat: Too many single-use plans consume memory.
- Pollution: Poorly parameterized queries flood cache with variations.

45. What are Trivial Plans vs Full Optimization Plans?

- Trivial Plan: Simple queries (SELECT * FROM small table).
- Full Optimization: Complex queries with joins, filters, aggregates.

46. What is a Plan Regression?

- When SQL Server chooses a worse plan after schema/statistics changes.
- Fix with Query Store Plan Forcing.

47. What are the differences between Estimated Cost Percentages in Execution Plan?

- They're optimizer estimates, not exact runtime CPU/IO usage.
- Use for relative comparison only.

48. How do you share Execution Plans?

- Save as .sqlplan file in SSMS.
- Export XML from DMVs (sys.dm_exec_query_plan).

49. What is a "Tipping Point" in Execution Plan?

- When optimizer switches from using index seeks to table/index scans.
- Usually happens when query returns large portion of rows.

50. How do you troubleshoot slow queries using Execution Plans?

- Steps:
    1. Capture Actual Execution Plan.
    2. Check for scans vs seeks.
    3. Look for key lookups, missing indexes, warnings.
    4. Compare Estimated vs Actual rows.
    5. Fix statistics/indexing/parameterization issues.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔷 Top 50: Query Optimization in SQL Server (DBA/Admin Focus)

General Concepts

1. What is query optimization in SQL Server?

- Process of improving query performance by minimizing CPU, IO, and memory usage.
- Done via indexing, statistics, rewriting queries, and optimizer hints.

2. What are common causes of slow queries?

- Missing or outdated indexes.
- Poor statistics.
- Non-SARGable predicates.
- Excessive joins or subqueries.
- Blocking and deadlocks.

3. How does the SQL Server Query Optimizer work?

- Generates multiple execution plans and picks the lowest estimated cost.
- Uses **Cardinality Estimator** to predict row counts.

4. What are the main steps to troubleshoot a slow query?

1. Capture execution plan.
2. Check estimated vs actual rows.
3. Identify expensive operators.
4. Look for missing indexes / key lookups.
5. Check waits (CXPACKET, LCK, etc.).

5. What is the difference between Logical and Physical Query Processing?

- Logical: Theoretical order of operations (FROM → WHERE → GROUP BY → SELECT → ORDER BY).
- Physical: Actual execution order chosen by SQL Server.

Indexes & Statistics

6. How do indexes help query optimization?

- Reduce I/O by allowing seeks instead of scans.
- Enable covering queries to avoid lookups.

7. How do you detect missing indexes?

- DMVs: sys.dm_db_missing_index_details.
- Execution Plan (Missing Index warning).

8. What are included columns in indexes and how do they help?

- Non-key columns stored at leaf level.
- Help queries become covering indexes.

9. What is index selectivity and why is it important?

- Ratio of unique values to total rows.
- High selectivity → better performance.

10. What role do statistics play in query optimization?

- Statistics guide optimizer to estimate row counts.
- Incorrect/outdated stats lead to bad plans.

11. How do you update statistics?

- UPDATE STATISTICS per table/index.
- sp_updatestats for database.
- AUTO_UPDATE_STATISTICS option recommended.

12. How do filtered indexes improve query optimization?

- Reduce index size and maintenance.
- Improve query selectivity when filtering common predicates.

Execution Plans

13. What is the difference between Index Seek and Index Scan?

- Seek: Uses key ranges (efficient).
- Scan: Reads all rows (less efficient).

14. How do you identify expensive operators in execution plans?

- Look for highest cost percentage.
- Use Actual Execution Plan for runtime details.

15. What does "Key Lookup" mean in execution plans?

- Non-clustered index missing required columns.
- Causes extra reads from clustered index/heap.

16. What does "Spill to TempDB" mean?

- Hash/Sort operator didn't get enough memory.
- Indicates query needs tuning or memory grant adjustments.

17. What is parameter sniffing and how does it affect query performance?

- First execution parameter creates cached plan.
- Other parameter values may cause suboptimal plans.
- Fix: OPTIMIZE FOR, RECOMPILE, plan guides.

18. What is a parallel execution plan?

- Uses multiple CPU threads.
- Controlled by MAXDOP and cost threshold for parallelism.

19. What is Cardinality Estimation?

- Process of predicting row counts.
- Wrong estimation = bad plans.

20. How do you detect plan regressions?

- Query Store (SQL 2016+).
- Compare old vs new plans.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Query Rewriting

21. Why are SARGable queries important?

- Queries that allow index usage (e.g., WHERE col = 5).
- Non-SARGable queries (e.g., WHERE YEAR(date) = 2023) force scans.

22. What's the impact of using SELECT *?

- Pulls unnecessary data.
- Increases I/O, memory, and network load.

23. How can rewriting subqueries into JOINs improve performance?

- JOINs are often more efficient than correlated subqueries.
- Reduces repeated lookups.

24. What's the difference between EXISTS vs IN vs JOIN for optimization?

- EXISTS: Best for checking existence.
- IN: Works well with small sets.
- JOIN: Useful for fetching related data.

25. Why should you avoid functions in WHERE clauses?

- Functions on columns (WHERE UPPER(name) = 'JOHN') prevent index usage.

26. How can CTEs (Common Table Expressions) help optimization?

- Improve readability.
- May help optimizer reuse plans better than deeply nested queries.

27. When should you use temporary tables vs table variables?

- Temp tables: Better with large datasets, support stats and indexing.
- Table variables: Good for small sets, no stats → risk of poor plan.

28. Why is UNION ALL often faster than UNION?

- UNION ALL skips duplicate elimination (no SORT step).

29. When should you use APPLY operators (CROSS APPLY/OUTER APPLY)?

- Useful for TVFs or correlated subqueries.
- Often better than nested loops.

30. Why should you avoid implicit conversions?

- Cause scans instead of seeks.
- Example: Comparing int column to varchar literal.

Performance Troubleshooting

31. What DMVs help identify slow queries?

- sys.dm_exec_query_stats
- sys.dm_exec_requests
- sys.dm_exec_query_plan

32. How do you use Query Store for optimization?

- Captures execution history.
- Compare plans, detect regressions, force stable plans.

33. What are Wait Statistics and how do they help optimization?

- Show where queries wait (CPU, IO, locks, memory).
- Helps find bottlenecks beyond T-SQL.

34. What is blocking and how does it affect query performance?

- When one query holds locks that block others.
- Causes long waits and slow execution.

35. What is deadlock and how does it impact queries?

- Circular blocking between two or more queries.
- One query is killed as a victim.

36. What's the impact of parameterized vs ad-hoc queries on optimization?

- Parameterized queries → plan reuse.
- Ad-hoc queries → plan cache bloat.

37. What is plan cache pollution?

- Too many single-use plans filling memory.
- Fix with Forced Parameterization or Optimize for Adhoc Workloads.

38. How do you troubleshoot queries spilling to TempDB?

- Check execution plan warnings.
- Increase memory grant, optimize joins, or add indexes.

39. How do you detect implicit conversions in queries?

- Execution plan warnings.
- DMV: sys.dm_exec_cached_plans with XML plan check.

40. What's the difference between Estimated and Actual Execution Plan?

- Estimated = optimizer guess.
- Actual = runtime values, includes row counts.

Advanced Optimization Techniques

41. How do table partitioning and partitioned indexes help optimization?

- Allow pruning of irrelevant partitions.
- Improve query speed for large tables.

---

42. What is Batch Mode execution and how does it help?

- Columnstore indexes allow batch processing of rows.
- Improves performance in analytics workloads.

43. What is Adaptive Query Processing?

- Features like Adaptive Joins, Memory Grant Feedback, Interleaved Execution.
- Adjusts execution during runtime.

44. How do you optimize queries with dynamic SQL?

- Parameterize where possible.
- Use sp_executesql for better plan reuse.

45. How do you handle skewed data distribution in query optimization?

- Use filtered stats.
- Add histogram steps.
- Query hints with OPTIMIZE FOR.

46. What is Query Hint "FORCESEEK" and when to use it?

- Forces index seek instead of scan.
- Use cautiously when optimizer misjudges.

47. How do you use indexed views for query optimization?

- Materialized result sets stored physically.
- Improve performance for aggregate-heavy queries.

48. What's the impact of locking hints like NOLOCK?

- Avoids blocking, but allows dirty reads.
- Faster but risky in financial/critical systems.

49. What is query compilation time and how does it affect performance?

- Time optimizer spends generating a plan.
- High compilation = bad parameterization or too many ad-hoc queries.

50. What best practices ensure long-term query optimization?

- Regular index maintenance.
- Update statistics.
- Monitor Query Store.
- Avoid SELECT *.
- Write SARGable queries.
- Capture and tune top resource-consuming queries regularly.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔷 Top 50: Wait Statistics & Blocking in SQL Server DBA

Wait Statistics – Concepts & Basics

1. What are wait statistics in SQL Server?

- SQL Server tracks why worker threads are waiting (CPU, IO, locks, memory).
- Helps identify performance bottlenecks.

2. Where are wait statistics stored?

- DMV: sys.dm_os_wait_stats.
- Aggregated since last SQL Server restart or DBCC SQLPERF("sys.dm_os_wait_stats", CLEAR).

3. Why are wait statistics important for a DBA?

- Show the *biggest bottlenecks* in the system.
- Guide tuning: CPU, IO, locking, memory pressure.

4. What are benign waits?

- Normal/harmless waits (e.g., WAITFOR, BROKER_RECEIVE_WAITFOR).
- Should be ignored.

5. What are signal waits vs resource waits?

- Resource waits: Waiting on a resource (disk, lock, memory).
- Signal waits: Waiting to get CPU after resource is available.

Common Wait Types

6. What is CXPACKET / CXCONSUMER wait?

- Related to parallel queries.
- High values = skewed parallelism or wrong MAXDOP.

7. What is PAGEIOLATCH_XX wait?

- Waiting on data pages to be read from disk.
- Indicates slow storage or insufficient memory.

8. What is PAGELATCH_XX wait?

- Contention on in-memory pages (e.g., PFS, GAM, SGAM).
- Common with heavy inserts in tempdb.

9. What is WRITELOG wait?

- Waiting on transaction log writes.
- Indicates slow log disk or excessive autogrowth.

10. What is ASYNC_NETWORK_IO wait?

- Query waiting to send results to client.
- Usually client/network bottleneck, not SQL issue.

11. What is SOS_SCHEDULER_YIELD wait?

- Thread yielded CPU voluntarily.
- Indicates CPU pressure.

12. What is RESOURCE_SEMAPHORE wait?

- Query waiting for memory grant.
- Indicates memory pressure or bad estimates.

13. What is LCK_M_XX wait?

- Lock waits (shared, update, exclusive).
- Indicates blocking.

14. What is HADR_SYNC_COMMIT wait?

- AlwaysOn AG synchronous commit replica delay.
- High wait = network/replica bottleneck.

15. What is OLEDB wait?

- Waiting on linked server queries.
- Indicates remote server slowness.

16. What is BACKUPIO / BACKUPBUFFER wait?

- Backup or restore operation waiting on IO.
- Could be slow disk or overloaded backup target.

17. What is RESOURCE_SEMAPHORE_QUERY_COMPILE wait?

- Too many concurrent queries waiting for memory to compile.
- Indicates high ad-hoc workload.

18. What is THREADPOOL wait?

- No worker threads available to run query.
- Indicates max worker thread exhaustion.

19. What is LATCH_EX vs LATCH_SH?

- LATCH_EX = Exclusive latch on memory structure.
- LATCH_SH = Shared latch.
- High contention = tempdb/system metadata issues.

20. What is PREEMPTIVE_OS_AUTHENTICATION wait?

- Waiting on external OS call (e.g., authentication, file system).
- Indicates AD or Kerberos slowness.

Blocking – Concepts & Basics

21. What is blocking in SQL Server?

- When one transaction holds a lock and prevents others from accessing the same resource.

22. What's the difference between blocking and deadlock?

- Blocking: Queries wait until resource is released.
- Deadlock: Circular blocking — SQL Server kills one as victim.

23. How do you detect blocking in SQL Server?

- DMVs:
  - sys.dm_exec_requests
  - sys.dm_exec_sessions
- Tools: Activity Monitor, Extended Events.

24. What is head blocker?

- The first session in blocking chain.
- Resolving head blocker clears all blocking.

25. How do you identify blocking chains?

- Use sp_who2 or sys.dm_exec_requests (blocking_session_id).
- Extended Events for detailed analysis.

26. What are common causes of blocking?

- Long-running transactions.
- Missing indexes → scans/locks.
- Poor isolation level usage.
- Open transactions left idle.

27. What is lock escalation?

- SQL Server converts many row/page locks into a table lock.
- Can cause blocking for other queries.

28. How do isolation levels affect blocking?

- READ COMMITTED: Default, causes blocking.
- SNAPSHOT/READ COMMITTED SNAPSHOT: Reduces blocking via versioning.

29. How do you minimize blocking?

- Keep transactions short.
- Use proper indexes.
- Choose appropriate isolation level.

30. How do you capture blocking events historically?

- Extended Events (blocked_process_report).
- SQL Trace/Profiler (deprecated).

Troubleshooting Blocking

31. What is a deadlock graph?

- Visual representation of processes involved in a deadlock.
- Captured in Extended Events or Profiler.

32. How do you resolve blocking caused by missing indexes?

- Check execution plan.
- Add covering or filtered index.

33. How do you troubleshoot blocking in tempdb?

- Look for PAGELATCH waits.
- Fix with multiple tempdb data files.

34. What's the role of NOLOCK in reducing blocking?

- Allows reading uncommitted data (dirty reads).
- Reduces blocking but risks incorrect results.

35. What is optimistic concurrency in blocking management?

- Readers don't block writers (versioning).
- Achieved with SNAPSHOT isolation.

36. How do you identify session details from SPID in blocking?

- DMV: sys.dm_exec_sessions (login, program, host name).

37. What SQL command can you use to kill a blocking session?

- KILL <SPID>

38. What are blocking alerts and how to configure them?

- Alerts when blocking exceeds threshold.
- Implement with Agent Alerts or Extended Events.

39. What is write blocking?

- Insert/update/delete blocked by another transaction.
- Often due to missing indexes or long transactions.

40. What is read blocking?

- Select blocked by update/delete transaction.
- Fix with READ COMMITTED SNAPSHOT.

Best Practices & Advanced Scenarios

41. How do you clear wait statistics?

- DBCC SQLPERF("sys.dm_os_wait_stats", CLEAR)

42. How do you correlate waits with queries?

- Use sys.dm_exec_requests joined with waits and query text.

43. What's the relationship between Wait Stats and PerfMon counters?

- Waits = SQL internal waits.
- PerfMon = OS-level performance counters.

44. How do you differentiate between IO bottleneck vs blocking issue?

- IO waits = PAGEIOLATCH / WRITELOG.
- Blocking waits = LCK_M_XX.

45. How do you handle THREADPOOL waits?

- Reduce concurrent requests.
- Tune queries.
- Increase MAX WORKER THREADS (last resort).

46. How do you reduce WRITELOG waits?

- Place log file on fast disk (SSD).
- Pre-size log file to avoid autogrowth.

47. How do you troubleshoot CXPACKET waits?

- Check query parallelism.
- Adjust MAXDOP.
- Increase cost threshold for parallelism.

48. How do you use Query Store for blocking analysis?

- Query Store captures execution statistics.
- Combine with wait stats for bottleneck diagnosis.

49. What is "headroom analysis" in wait stats?

- Checking which waits dominate overall time.
- Helps prioritize tuning efforts.

50. What are the best practices for wait stats & blocking monitoring?

- Baseline normal waits.
- Automate periodic wait stats collection.
- Enable Extended Events for blocking chains.
- Proactively tune queries/indexes.
- Use optimistic concurrency where applicable.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🛠 SQL Server Wait Statistics Troubleshooting Guide

1. CPU / Scheduler Related

- **Wait Type:** SOS_SCHEDULER_YIELD
  → **Root Cause:** High CPU usage, poor query plans, excessive parallelism.
  → **Action:**
    - Identify top CPU-consuming queries (sys.dm_exec_query_stats).
    - Review indexes and missing statistics.
    - Consider adjusting MAXDOP and Cost Threshold for Parallelism.

2. Locking / Blocking

- **Wait Type:** LCK_M_X, LCK_M_S, LCK_M_U
  → **Root Cause:** Long-running transactions holding locks, blocking chains.
  → **Action:**
    - Use sp_whoisactive or Activity Monitor to find blockers.
    - Break blocking chain (commit/rollback).
    - Optimize queries causing escalation.
    - Consider snapshot isolation / RCSI.

3. I/O Related

- **Wait Type:** PAGEIOLATCH_SH, PAGEIOLATCH_EX
  → **Root Cause:** Slow storage subsystem, heavy read/write workload, missing indexes.
  → **Action:**
    - Check disk latency via sys.dm_io_virtual_file_stats.
    - Add indexes to reduce table scans.
    - Move TempDB / data files to faster storage.
    - Consider buffer pool extension / memory increase.

4. Memory Related

- **Wait Type:** RESOURCE_SEMAPHORE
  → **Root Cause:** Query memory grants too high, insufficient memory, bad estimates.
  → **Action:**
    - Identify queries waiting on memory grants (sys.dm_exec_query_memory_grants).
    - Tune queries / indexes for better estimates.
    - Reduce max server memory if OS is starving.
    - Upgrade RAM if persistent.

5. Network / External Calls

- **Wait Type:** ASYNC_NETWORK_IO
  → **Root Cause:** Client app slow to consume results (not SQL itself).
  → **Action:**
    - Check app fetching logic (e.g., row-by-row instead of set-based).
    - Optimize result set size (return fewer rows).
    - Use batching instead of single fetch loops.

6. Parallelism

- **Wait Type:** CXPACKET (pre-SQL 2016), CXCONSUMER (post-SQL 2016)
  → **Root Cause:** Queries running in parallel with imbalance, skewed workloads.
  → **Action:**
    - Review query plans for parallel operators.
    - Tune queries to avoid unnecessary parallelism.
    - Adjust MAXDOP and Cost Threshold for Parallelism.
    - Ensure stats are updated for balanced distribution.

7. TempDB Bottlenecks

- **Wait Type:** PAGELATCH_UP, PAGELATCH_EX (on TempDB)
  → **Root Cause:** Allocation contention in TempDB (system tables, temp objects).
  → **Action:**
    - Configure multiple TempDB data files (1 per 2–4 cores).
    - Enable trace flag 1118 (pre-2016, default in 2016+).
    - Monitor TempDB usage.

8. External Resource

- **Wait Type:** OLEDB, PREEMPTIVE_XXX waits
  → **Root Cause:** Linked servers, external backups, or OS-level operations.
  → **Action:**
    - Optimize linked server queries.
    - Check external calls (backups, file I/O).
    - Reduce reliance on external dependencies.

High-Level Flow (Interview-Ready)

[Check sys.dm_os_wait_stats]
|
v
[Identify Top Wait Type] → [Map to Category]
|
v
[Root Cause Analysis] → [Corrective Action]

- CPU waits → Tune queries, indexes, parallelism.
- Lock waits → Resolve blocking, tune transactions.
- I/O waits → Improve storage, indexing.
- Memory waits → Fix grants, upgrade RAM.
- Network waits → Optimize app fetch logic.
- TempDB waits → Add files, reduce contention.
- External waits → Fix linked servers / OS bottlenecks.

📊 Top 50 – Statistics & Cardinality in SQL Server (DBA-Focused)

1. **What are statistics in SQL Server and why are they important?**
   - Statistics describe data distribution (histogram + density) and help the optimizer estimate row counts.
2. **Where are statistics stored?**
   - In system tables within each database (sys.stats, sys.stats_columns), and histograms are in DBCC SHOW_STATISTICS.
3. **What is a histogram in statistics?**
   - Shows distribution of column values in up to 200 steps. Used for cardinality estimation.
4. **What is density in SQL Server statistics?**
   - Reciprocal of distinct values (1 / NDV). Helps with estimating selectivity for predicates.
5. **What is cardinality estimation?**
   - The process SQL Server uses to predict the number of rows returned from a query.
6. **Why is cardinality estimation important?**
   - Incorrect estimates → bad plans → performance issues (wrong joins, spills, scans instead of seeks).
7. **What is the Cardinality Estimator (CE) in SQL Server?**
   - A model introduced in SQL 7.0, enhanced in SQL 2014 and later, that predicts row counts using statistics.
8. **Difference between Legacy CE (SQL 2012 and earlier) and New CE (SQL 2014+)?**
   - New CE assumes less correlation between predicates, producing different estimates.
9. **How to check which CE version is being used?**
   - Use DBCC TRACEON(9481) for legacy or DBCC TRACEON(2312) for new CE; query plan shows CE version.
10. **What happens when statistics are missing or stale?**
    - Optimizer guesses row counts → bad plans → performance degradation.
11. **What triggers automatic statistics creation?**
    - AUTO_CREATE_STATISTICS database option (ON by default).
12. **What triggers automatic statistics updates?**
    - AUTO_UPDATE_STATISTICS (ON by default). Updates when ~20% + 500 rows changed.
13. **What is AUTO_UPDATE_STATISTICS_ASYNC?**
    - Updates statistics in background, query uses old stats until new stats are ready.
14. **How to manually update statistics?**
    - UPDATE STATISTICS table(column) or sp_updatestats.
15. **What's the difference between UPDATE STATISTICS and sp_updatestats?**
    - UPDATE STATISTICS = specific, sp_updatestats = database-wide with minimal overhead.
16. **What are filtered statistics?**
    - Stats on subset of rows (with a filter predicate). Helps with skewed data.
17. **What are incremental statistics?**
    - For partitioned tables (Enterprise Edition). Updates only changed partitions.
18. **What is the impact of statistics on parameter sniffing?**
    - Statistics influence compiled plans; skewed data can cause suboptimal reuse.
19. **How to detect out-of-date statistics?**
    - DMV: sys.dm_db_stats_properties (last_updated).
20. **How to force SQL Server to ignore statistics?**
    - Use query hints like OPTION (OPTIMIZE FOR UNKNOWN) or disable auto-stats.
21. **When should you disable auto-create or auto-update statistics?**
    - Rarely; mainly for very large ETL systems with custom stat management.
22. **What is sampled statistics vs full scan?**
    - WITH SAMPLE reads subset of rows; WITH FULLSCAN scans all rows → more accurate but expensive.
23. **How do skewed data distributions affect cardinality estimation?**
    - Estimates may be wrong if stats don't reflect skew. Use filtered stats or histograms.

24. **What is ascending key problem in statistics?**
    - New values beyond histogram max are estimated poorly (seen in identity/date columns).
25. **How to solve the ascending key problem?**
    - Use trace flags, filtered stats, or frequent manual updates.
26. **What's the difference between statistics created by indexes vs auto-created statistics?**
    - Index stats are maintained automatically with index updates; auto stats are per-column only.
27. **Can statistics exist without an index?**
    - Yes, auto-created or manually created statistics.
28. **How to view statistics object details?**
    - DBCC SHOW_STATISTICS (table, stat_name).
29. **What is histogram step compression?**
    - If >200 unique values, SQL Server compresses values into max 200 steps.
30. **How does SQL Server estimate joins?**
    - Uses histograms and density vectors; assumes independence by default.
31. **What are multi-column statistics?**
    - Stats across multiple columns in one object; helps with correlated predicates.
32. **What are limitations of multi-column statistics?**
    - Only the leftmost column has histogram; others rely on density.
33. **How do outdated statistics affect parallelism?**
    - Wrong row count estimates can cause bad DOP (degree of parallelism) decisions.
34. **How do statistics affect TempDB usage?**
    - Wrong estimates can cause spills to TempDB due to underestimated memory grants.
35. **How can statistics affect plan stability in Always On AGs?**
    - Statistics are database-scoped; secondary replicas don't auto-update stats.
36. **What happens to statistics after a database restore?**
    - They restore as-is; may be outdated relative to new workload.
37. **What is the difference between persisted sample statistics and temporary estimates?**
    - Persisted stats are stored objects; estimates can come from heuristics if missing.
38. **How can statistics cause recompilation?**
    - When stats are updated, plans depending on them may recompile.
39. **What is statistics blob in execution plan?**
    - Plan XML contains stats ID and histogram reference.
40. **What is the impact of statistics on columnstore indexes?**
    - Columnstore segment elimination relies on metadata; stats still guide joins/filters.
41. **How to detect queries suffering from poor cardinality estimation?**
    - Compare EstimatedRows vs ActualRows in execution plans.
42. **What tools help monitor statistics health?**
    - DMVs, Query Store, Extended Events, Plan Explorer.
43. **What happens if statistics are dropped?**
    - Optimizer loses row distribution info → poor plan selection.
44. **How often should statistics be updated in a VLDB?**
    - Depends on workload; often schedule nightly or incremental updates.
45. **What is trace flag 2371?**
    - Lowers auto-update threshold for stats on large tables (default since SQL 2016).
46. **What is "statistics object auto-naming"?**
    - SQL Server creates names like _WA_Sys_<hex> for auto-created stats.
47. **How does cardinality estimation handle OR predicates?**
    - CE may overestimate row counts by assuming independence.

48. **How does cardinality estimation handle AND predicates?**
    - CE may underestimate by multiplying selectivities.
49. **What is the difference between scalar UDFs and inline table functions regarding statistics?**
    - Scalar UDFs are opaque → no stats, inline TVFs expose query → better estimates.
50. **What are best practices for managing statistics?**
    - Keep auto-stats ON, use FULLSCAN for skewed data, monitor regularly, use filtered/incremental stats as needed.

🧠 Top 50 – Memory & TempDB (SQL Server DBA Administration)

🔷 Memory Management

1. **How does SQL Server use memory?**
    - Buffer pool (data cache), plan cache, columnstore pool, in-memory OLTP, and workspace memory (sort/hash).
2. **What is the buffer pool?**
    - Main memory cache where SQL Server stores data and index pages after first read.
3. **What is the role of the Lazy Writer process?**
    - Frees up dirty or unused pages from the buffer pool when memory pressure occurs.
4. **What is the role of the Checkpoint process?**
    - Writes dirty pages from memory to disk periodically, reducing recovery time.
5. **How do you configure Max Server Memory?**
    - Set via sp_configure to prevent SQL Server from consuming all OS memory.
6. **What happens if Max Server Memory is not set?**
    - SQL Server may consume all RAM, starving the OS and causing performance issues.
7. **What is Min Server Memory?**
    - The floor memory SQL Server tries to retain once allocated (not a guaranteed reservation).
8. **How to monitor memory usage?**
    - DMVs: sys.dm_os_process_memory, sys.dm_os_memory_clerks, Perfmon counters.
9. **What is the difference between memory clerks and memory consumers?**
    - Clerks = internal accounting units, consumers = objects using memory (caches, pools).
10. **What is memory pressure in SQL Server?**
    - When available memory drops below required, forcing evictions from caches.
11. **What is external vs internal memory pressure?**
    - External: OS or other processes consume RAM. Internal: SQL Server subsystems demand more memory than allocated.
12. **How to detect memory pressure?**
    - Look for PAGEIOLATCH waits, low buffer cache hit ratio, frequent evictions in DMV.
13. **What is a stolen page?**
    - Memory page used for workspace (sort/hash) instead of data cache.
14. **What is single-page allocator vs multi-page allocator?**
    - Single: <8KB pages (inside buffer pool). Multi: >8KB allocations (outside buffer pool, removed in SQL 2012+).
15. **What is Columnstore memory usage?**
    - Uses Columnstore Object Pool (separate from buffer pool) for segment caching.
16. **What is Resource Governor?**
    - Controls CPU, memory grants, and I/O per workload group.
17. **What are memory grants in SQL Server?**
    - Memory requested by queries for sort, hash, or joins before execution.
18. **What are memory grant waits?**
    - Queries waiting for workspace memory → can cause RESOURCE_SEMAPHORE waits.

19. **What is the Query Memory Grant Feedback feature?**
    - Adjusts memory grant estimates dynamically (introduced in SQL 2017, enhanced later).
20. **What causes excessive memory grants?**
    - Bad statistics, poor cardinality estimation → inflated row counts.
21. **How to fix memory grant issues?**
    - Update stats, use OPTION (MAX_GRANT_PERCENT) or tune queries.
22. **What is In-Memory OLTP memory behavior?**
    - Uses separate memory space, not part of buffer pool. Must plan capacity carefully.
23. **What is locked pages in memory (LPIM)?**
    - Prevents OS from paging out SQL Server memory. Recommended for dedicated servers.
24. **When would you not enable LPIM?**
    - On shared servers where OS and other services need memory flexibility.
25. **How to diagnose memory leaks in SQL Server?**
    - Check sys.dm_os_memory_clerks growth, correlate with workload, use Extended Events.

🔷 TempDB Management

26. **What is TempDB used for?**
    - System database for temp tables, table variables, worktables, version store, sorting, hashing, row versioning.
27. **Why is TempDB performance critical?**
    - High concurrency queries (joins, sorts, temp objects) heavily depend on TempDB.
28. **What is TempDB contention?**
    - Multiple sessions contending for PFS, GAM, SGAM pages → latch contention.
29. **How to detect TempDB contention?**
    - PAGELATCH_UP/EX waits on TempDB pages (not PAGEIOLATCH).
30. **How to fix TempDB contention?**
    - Add multiple TempDB data files (equal size), enable TF 1118 (or SQL 2016+ defaults).
31. **How many TempDB data files should be created?**
    - Start with 1/4 to 1/2 of logical CPU cores (max ~8), adjust by monitoring contention.
32. **Should TempDB files all be the same size?**
    - Yes, to balance proportional fill across files.
33. **What is the role of TempDB log file?**
    - Logs transactions in TempDB (despite auto-truncation). Still needed for recovery.
34. **What is the version store in TempDB?**
    - Stores row versions for snapshot isolation and online index rebuilds.
35. **What happens if version store grows too large?**
    - Can fill TempDB, causing snapshot isolation errors.
36. **How to monitor version store usage?**
    - DMV: sys.dm_tran_version_store_space_usage.
37. **What is TempDB spill?**
    - When query exceeds memory grant, intermediate results spill to TempDB (sort/hash spill).
38. **How to detect TempDB spills?**
    - Execution plan shows warnings (yellow exclamation mark), or spill_level in XE.
39. **How to reduce TempDB spills?**
    - Increase memory grants, tune queries, improve statistics.
40. **What is TF 1117 and 1118 for TempDB?**
    - TF 1117 = grow all files together, TF 1118 = uniform extent allocation. Default since SQL 2016.
41. **Why place TempDB on fast storage (SSD/NVMe)?**

- High I/O workload, frequent writes, random access.

42. **Should TempDB use autogrowth?**
    - Yes as a safety net, but pre-size files to avoid frequent growth events.
43. **How to monitor TempDB space usage?**
    - DMV: sys.dm_db_file_space_usage.
44. **What is the impact of table variables on TempDB?**
    - Stored in TempDB (contrary to common belief). Still generate I/O and contention.
45. **What is the impact of temp tables on performance?**
    - Require stats generation, can be indexed, more optimizer-friendly than table variables.
46. **What's the difference between table variables and temp tables in TempDB usage?**
    - Table variables = fewer recompiles, no stats (pre-SQL 2019). Temp tables = stats, better estimates.
47. **How did SQL 2019 improve table variables?**
    - Introduced deferred compilation → better cardinality estimates.
48. **How can TempDB cause blocking issues?**
    - If version store cleanup lags or TempDB log fills up.
49. **What are best practices for configuring TempDB?**
    - Multiple equal files, trace flags (or defaults in SQL 2016+), pre-size files, fast storage.
50. **What are common wait types related to TempDB?**
    - PAGELATCH_* (contention), WRITELOG (log bottleneck), RESOURCE_SEMAPHORE (spills).

Top 50 – I/O & Disk Optimization in SQL Server (DBA Administration)

I/O Concepts & Internals

1. **How does SQL Server perform I/O operations?**
    - Uses Windows APIs for reads/writes, with pages (8 KB) as the basic unit.
2. **What is a data page in SQL Server?**
    - 8 KB structure storing rows, allocated in extents (8 contiguous pages = 64 KB).
3. **What are extents in SQL Server?**
    - 8-page units; can be mixed (different objects) or uniform (single object).
4. **What is write-ahead logging (WAL)?**
    - SQL Server writes log records to disk before dirty pages are flushed.
5. **What is checkpoint I/O?**
    - Writes dirty pages to disk periodically to reduce recovery time.
6. **What is lazy writer I/O?**
    - Background process writing least-used dirty pages when memory pressure occurs.
7. **What is indirect checkpoint?**
    - Database-level setting that controls recovery time (seconds), flushing dirty pages accordingly.
8. **What is sequential vs random I/O in SQL Server?**
    - Log writes = sequential; data page reads/writes = random.
9. **Why are transaction log files write-intensive?**
    - Every modification must be logged sequentially.
10. **How does SQL Server use TempDB I/O?**
    - For temp tables, spills, version store, hash/sort operations → very write-heavy.

I/O Bottlenecks & Troubleshooting

11. **How do you detect I/O bottlenecks?**

- Perfmon counters (Disk sec/Read, Disk sec/Write), DMVs (sys.dm_io_virtual_file_stats).

12. **What is a page latch vs page I/O latch wait?**
    - PAGELATCH_* = in-memory contention; PAGEIOLATCH_* = waiting on physical I/O.
13. **What are common I/O-related wait types?**
    - PAGEIOLATCH_*, WRITELOG, ASYNC_IO_COMPLETION, IO_COMPLETION.
14. **What is the impact of WRITELOG waits?**
    - Transaction log flush is slow → bottleneck in commit operations.
15. **How do you differentiate between storage latency and SQL-level bottlenecks?**
    - Compare OS counters with SQL DMVs for correlation.
16. **How do you measure log file I/O latency?**
    - sys.dm_io_virtual_file_stats → check io_stall_write_ms / num_of_writes.
17. **How do you measure data file I/O latency?**
    - Same DMV, focusing on read/write stalls for MDF/NDF files.
18. **What is Instant File Initialization (IFI)?**
    - Allows SQL Server to skip zeroing data files, speeding up allocations (not log files).
19. **How do you enable Instant File Initialization?**
    - Grant "Perform volume maintenance tasks" right to SQL Server service account.
20. **What happens without IFI?**
    - File growth/restores are slower because space must be zeroed.

🔷 Disk & Storage Optimization

21. **What are best practices for database file placement?**
    - Separate data, log, TempDB, and backups across different volumes when possible.
22. **Why should transaction log files be isolated on separate disks?**
    - Sequential write patterns → better throughput when isolated.
23. **Why should TempDB be placed on fast storage (SSD/NVMe)?**
    - High concurrency workloads → reduces latch contention & spills.
24. **What is the recommended RAID configuration for data files?**
    - RAID 10 for performance and fault tolerance; RAID 5/6 only if budget constraints.
25. **What is the recommended RAID configuration for log files?**
    - RAID 10 (write-heavy, sequential I/O).
26. **Why are multiple data files recommended for TempDB?**
    - To reduce allocation contention across PFS, GAM, SGAM.
27. **How to size database files for optimal I/O?**
    - Pre-size files based on expected workload, avoid frequent autogrowth.
28. **What's the problem with small autogrowth increments?**
    - Leads to file fragmentation and repeated growth overhead.
29. **What's the problem with very large autogrowth increments?**
    - May freeze queries temporarily during allocation.
30. **How to monitor physical disk throughput?**
    - Perfmon: Disk Reads/sec, Writes/sec, Disk Transfers/sec.

🔷 Advanced I/O Optimization

31. **What is Read-Ahead I/O?**
    - SQL Server pre-fetches pages into memory to speed up sequential scans.
32. **What is checkpoint throttling?**
    - Controls checkpoint write rates to prevent overwhelming I/O subsystem.

33. **What are indirect checkpoints' benefits for I/O?**
    - Provides predictable recovery time, smooths out I/O spikes.
34. **What is delayed durability?**
    - Allows log writes to be batched asynchronously, reducing WRITELOG waits.
35. **When would you enable delayed durability?**
    - High-throughput, low-criticality workloads (risk: data loss if crash before flush).
36. **What's the impact of FILESTREAM on I/O?**
    - Moves large unstructured data to NTFS, bypassing buffer pool.
37. **What's the impact of Columnstore indexes on I/O?**
    - Reduce I/O footprint via compression, segment elimination.
38. **What's the role of buffer cache hit ratio in I/O optimization?**
    - Higher ratio = fewer physical I/Os → workload fits in memory.
39. **How does SQL Server decide between physical I/O and memory?**
    - Checks buffer pool first; if page not found, triggers physical I/O.
40. **How can Resource Governor help with I/O?**
    - Limits I/O for workloads via MAX_IOPS_PER_VOLUME.

🔷 Backup, Restore & I/O

41. **What's the I/O pattern of backups?**
    - Large sequential reads from data files, writes to backup file.
42. **What's the I/O pattern of restores?**
    - Writes to data/log files, reads backup file.
43. **How does backup compression impact I/O?**
    - Reduces I/O volume, increases CPU usage.
44. **What's the role of striping in backups?**
    - Parallel writes across multiple disks → improves throughput.
45. **How to troubleshoot slow backups/restores?**
    - Measure backup throughput, check storage latency, compression effects.

🔷 Best Practices & Real-Time Scenarios

46. **What is the recommended latency target for OLTP systems?**
    - <1 ms for log writes, <5 ms for random reads/writes.
47. **What is the recommended latency target for OLAP systems?**
    - Higher tolerance (~10–20 ms), since queries are read-heavy.
48. **What's the impact of virtualization on SQL I/O performance?**
    - Shared storage can introduce noisy neighbor effects.
49. **How to reduce I/O hot spots?**
    - Partitioning, spreading files across volumes, SSDs.
50. **What are the golden rules of SQL I/O optimization?**
    - Pre-size files, use IFI, isolate logs, use fast disks for TempDB, monitor latency continuously.

📈 Top 50 – Performance Monitoring Tools in SQL Server (DBA Administration)

🔷 Native SQL Server Tools

1. **SQL Server Management Studio (SSMS) – Activity Monitor**
   - Quick overview: CPU, I/O, recent expensive queries.
2. **SQL Profiler**
   - Legacy tracing tool for capturing query activity (deprecated but still used).
3. **Extended Events (XEvents)**
   - Lightweight replacement for Profiler, captures detailed query & engine events.
4. **SQL Trace**
   - Old tracing engine (predecessor to XEvents).
5. **Query Store**
   - Captures execution plans, runtime stats, regressions, and allows plan forcing.
6. **Database Engine Tuning Advisor (DTA)**
   - Suggests indexes and partitions (caution: validate before implementing).
7. **Performance Dashboard Reports (SSMS add-in)**
   - Visual reports for wait stats, I/O, expensive queries.
8. **Dynamic Management Views (DMVs)**
   - Expose internal runtime, I/O, waits, execution statistics.
9. **SQL Server Error Logs**
   - For startup, error, deadlock, and blocking event monitoring.
10. **SQL Server Agent Alerts**

- Automates alerts for job failures, severity-level errors, performance conditions.

🔷 Key DMV-based Monitoring

11. **sys.dm_exec_requests**

- Shows current running requests, waits, blocking sessions.

12. **sys.dm_exec_sessions**

- Tracks login sessions, CPU/memory usage.

13. **sys.dm_exec_query_stats**

- Aggregated execution statistics for cached queries.

14. **sys.dm_exec_sql_text()**

- Retrieves SQL text for query handles.

15. **sys.dm_exec_query_plan()**

- Returns cached execution plans.

16. **sys.dm_db_index_usage_stats**

- Reveals index usage patterns (seeks, scans, lookups).

17. **sys.dm_db_index_operational_stats**

- Low-level I/O and lock contention stats.

18. **sys.dm_os_wait_stats**

- Key DMV for analyzing wait-based performance tuning.

19. **sys.dm_io_virtual_file_stats**

- File-level I/O stall times and throughput.

20. **sys.dm_exec_procedure_stats**

- Aggregated stats for stored procedure performance.

Query Performance & Plan Monitoring

21. **Execution Plan Viewer (SSMS)**
- Displays estimated vs actual query plans.
22. **Live Query Statistics**
- Real-time view of query execution progress.
23. **Query Performance Insight (Azure SQL)**
- Cloud-native monitoring of top queries.
24. **SET STATISTICS TIME / IO**
- Shows per-query CPU time and logical/physical reads.
25. **sys.dm_exec_query_store_plan**
- Retrieve query plans stored in Query Store.
26. **sys.dm_exec_query_store_runtime_stats**
- Historical execution runtime info.
27. **sys.dm_exec_cached_plans**
- Monitor plan cache usage & evictions.
28. **Plan Cache (sys.dm_exec_plan_attributes)**
- Helps detect parameter sniffing and plan reuse issues.
29. **sp_BlitzCache (First Responder Kit)**
- Community tool for top CPU/I/O/wait queries.
30. **sp_BlitzWho**
- Realtime equivalent of Activity Monitor, lightweight.

Windows / OS Tools

31. **Performance Monitor (PerfMon)**
- Tracks counters like Disk I/O, Memory, Page Life Expectancy, Lazy Writes.
32. **Resource Monitor**
- Windows tool showing live CPU, memory, disk usage.
33. **Task Manager**
- Quick CPU/memory snapshot (not detailed for SQL).
34. **Event Viewer**
- Logs system, application, and SQL-related errors.
35. **Windows Perf Counters in SQL**
- Exposed in DMVs, e.g., sys.dm_os_performance_counters.
36. **Process Explorer (Sysinternals)**
- Advanced task manager for process-level memory/I/O breakdowns.
37. **ProcMon (Sysinternals)**
- Low-level monitoring of file and registry access.
38. **Diskspd / SQLIO**
- Microsoft tools for testing storage subsystem throughput.
39. **Latency Monitoring Tools (e.g., Storage vendor tools)**
- Helps correlate SQL stalls with SAN/VM storage issues.
40. **Resource Governor (SQL feature)**
- Monitors/limits workload CPU & memory.

Cloud & Third-Party Tools

41. **Azure Monitor (Log Analytics)**
- Tracks SQL metrics across Azure SQL, VM-hosted SQL.

42. **SQL Insights (Azure SQL)**
- Performance metrics collection & visualization.

43. **Redgate SQL Monitor**
- Popular third-party tool for real-time monitoring, baselining, and alerting.

44. **SolarWinds DPA (Database Performance Analyzer)**
- Wait-time focused performance analysis.

45. **SentryOne SQL Sentry**
- Query monitoring, blocking analysis, alerting.

46. **Idera SQL Diagnostic Manager**
- Performance dashboard, query analytics, alerting.

47. **Quest Foglight for SQL Server**
- Enterprise monitoring with correlation to infrastructure.

48. **Spotlight on SQL Server (Quest)**
- Visual diagnostics for waits, I/O, deadlocks.

49. **Dynatrace / AppDynamics / New Relic**
- APM tools that monitor SQL performance in application context.

50. **dbForge Monitor (Devart)**
- Free monitoring add-in for SSMS (real-time sessions, waits, I/O).

Here's a **comparison matrix** that maps each **Performance Monitoring Tool → Purpose → When to Use**, so you can **answer scenario-based interview questions quickly**.

📊 SQL Server Performance Monitoring – Comparison Matrix

Tool	Purpose	When to Use
SSMS Activity Monitor	Quick snapshot of resource usage & top queries	For fast health checks (not deep analysis)
SQL Profiler	Capture query activity (deprecated)	Legacy environments still using it
Extended Events (XEvents)	Lightweight tracing of query & engine events	Modern replacement for Profiler
SQL Trace	Old tracing mechanism	Only if dealing with legacy systems
Query Store	Captures execution plans & runtime stats	To detect plan regressions or force plans
DTA (Tuning Advisor)	Suggests indexes/partitions	Initial index tuning (validate output manually)
Performance Dashboard Reports	Graphical SSMS reports	Quick insights without custom queries
DMVs	Runtime/session/IO/waits stats	Always (core of DMV-based monitoring)
Error Logs	Log errors, deadlocks, startup issues	Post-incident troubleshooting
SQL Agent Alerts	Proactive alerts for failures/performance	Automated monitoring & notification

🔷 DMV-Focused Tools

DMV	Purpose	When to Use
sys.dm_exec_requests	Running requests, waits, blocking	To find current blocking/query issues
sys.dm_exec_sessions	CPU/memory per session	Session-level resource tracking
sys.dm_exec_query_stats	Aggregated execution stats	To find expensive queries
sys.dm_exec_sql_text()	Query text retrieval	Identifying problematic SQL
sys.dm_exec_query_plan()	Execution plan lookup	Deep dive into query execution
sys.dm_db_index_usage_stats	Index usage patterns	To find unused/overused indexes
sys.dm_db_index_operational_stats	Locking & low-level I/O stats	Debugging contention/index impact
sys.dm_os_wait_stats	Wait analysis	Root cause of performance bottlenecks
sys.dm_io_virtual_file_stats	File-level I/O stalls	Troubleshooting slow disks/storage
sys.dm_exec_procedure_stats	Procedure runtime stats	For proc-level tuning

---

🔷 Query & Plan Analysis

Tool	Purpose	When to Use
Execution Plan Viewer	Estimated vs actual plans	Query optimization
Live Query Statistics	Real-time execution progress	Long-running query debugging
Query Performance Insight (Azure)	Top queries in Azure SQL	Cloud-native workloads
SET STATISTICS TIME/IO	CPU, reads per query	Quick tuning in dev/test
Query Store DMVs	Plan/runtime history	Regression troubleshooting
Plan Cache DMVs	Detect plan reuse issues	Parameter sniffing analysis
sp_BlitzCache	Top resource queries	Community-driven query healthcheck
sp_BlitzWho	Lightweight session monitor	Real-time blocking/wait diagnosis

🔷 OS & Windows Tools

Tool	Purpose	When to Use
PerfMon	Counters for CPU, I/O, Memory	Long-term baselining
Resource Monitor	Live OS usage	Correlating SQL vs OS usage
Task Manager	Quick glance CPU/memory	Basic checks only
Event Viewer	Error/event logs	OS, SQL crash, or service failures
sys.dm_os_performance_counters	SQL Perf counters	In-SQL monitoring
Process Explorer (Sysinternals)	Detailed process info	Memory/I/O leaks
ProcMon (Sysinternals)	File/registry activity	Debugging SQL/OS interactions
Diskspd / SQLIO	Disk throughput tests	Benchmarking storage performance
Storage Vendor Tools	Latency/deep I/O stats	SAN/VM storage troubleshooting
Resource Governor	CPU/memory workload limits	Preventing runaway workloads

🔷 Third-Party & Cloud Tools

Tool	Purpose	When to Use
Azure Monitor	Cross-service monitoring	Azure hybrid/cloud
SQL Insights (Azure)	Collects DB metrics	Cloud-based DBA monitoring
Redgate SQL Monitor	Real-time dashboards & alerts	Popular 3rd-party tool for production
SolarWinds DPA	Wait-time analysis	Enterprise SQL farms
SentryOne SQL Sentry	Blocking, alerts	High concurrency environments
Idera SQL DM	Performance dashboard	Mid-large enterprise DBAs
Quest Foglight	Full infra + SQL monitoring	End-to-end monitoring
Spotlight (Quest)	Visual wait diagnostics	Quick problem detection
Dynatrace/AppDynamics/New Relic	APM with SQL integration	App + DB correlation
dbForge Monitor	Free SSMS add-in	Lightweight SQL monitoring

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

⚡ Top 50 Optimization Scenarios – SQL Server DBA Administration

🔷 Query & Execution Plan Optimization

1. Query performing table scans → Add covering index or rewrite joins.
2. Slow query due to parameter sniffing → Use OPTIMIZE FOR, recompile, or plan guides.
3. Non-SARGable WHERE clause (LIKE '%abc', ISNULL()) → Rewrite predicates.
4. Poor join performance → Ensure proper join keys, indexed joins.
5. Missing statistics → Update stats with FULLSCAN.
6. Stale statistics → Enable AUTO_UPDATE_STATISTICS_ASYNC.
7. Overly complex query with nested views → Flatten logic, use temp tables.
8. Scalar UDF bottleneck → Replace with inline table-valued functions.
9. Overuse of cursors → Replace with set-based queries.
10. Sorting bottleneck → Add index to support ORDER BY.

🔷 Indexing Optimization

11. Missing indexes identified in execution plan → Create optimal non-clustered indexes.
12. Duplicate/overlapping indexes → Consolidate to reduce maintenance overhead.
13. Over-indexed table → Drop unused indexes (sys.dm_db_index_usage_stats).
14. Heap table with frequent lookups → Add clustered index.
15. Wide clustered index key → Redesign to reduce row size & I/O.
16. Incorrect clustered index (on volatile column) → Rebuild on stable column.
17. Non-clustered index not covering query → Add included columns.
18. High fragmentation → Use index rebuild/reorganize with fill factor.
19. Partitioned table → Align indexes with partitioning key.
20. Filtered indexes → Use when queries filter on selective predicates.

🔷 Memory & CPU Optimization

21. High CPU usage from bad query plans → Force good plans with Query Store.
22. Excessive recompilations → Parameterize queries.
23. Insufficient memory grants → Tune queries, increase max server memory.
24. Low Page Life Expectancy (PLE) → Add memory or optimize buffer usage.
25. Excessive parallelism (CXPACKET waits) → Adjust MAXDOP, cost threshold.
26. Serial plan for large query → Ensure statistics/indexes allow parallelism.
27. High compilation time → Optimize schema, reuse query plans.
28. High context switching → Balance workloads, tune CPU affinity.
29. NUMA imbalance → Configure SQL NUMA node affinity properly.
30. Memory pressure from large objects (LOBs) → Use FILESTREAM, optimize schema.

🔷 I/O & TempDB Optimization

31. High PAGEIOLATCH waits → Improve disk subsystem or caching.
32. Frequent disk spills (workfile usage) → Increase memory, optimize queries.
33. Hotspot contention in TempDB → Add multiple TempDB files.
34. TempDB GAM/SGAM contention → Enable trace flag 1118 (pre-SQL 2016).
35. TempDB autogrowth events → Pre-size TempDB to required size.
36. Large table scans → Use partitioning, filtered indexes.
37. High I/O latency from log writes → Place transaction log on fast disk.
38. Read-heavy workload → Use read-only replicas / secondary indexes.

39. Write-heavy workload → Optimize batch commits, table partitioning.
40. Frequent checkpoint I/O spikes → Tune recovery interval, stagger workloads.

🔷 **Concurrency & Locking Optimization**

41. Blocking chain → Identify head blocker with DMVs, tune query.
42. Deadlocks → Add proper indexes, reduce transaction scope.
43. Long transactions → Break into smaller commits.
44. Lock escalation (row → table) → Use hints (ROWLOCK) or redesign queries.
45. Temp table vs table variable choice → Use temp tables with stats when large.
46. Snapshot isolation → Enable RCSI to reduce blocking.
47. High WRITELOG waits → Batch inserts/updates.
48. Reader-writer contention → Use partitioning, optimize access pattern.
49. High lock waits → Tune transaction isolation level.
50. Batch vs singleton operations → Switch to set-based operations.

❓ **Top 50 Advanced Optimization Scenarios – FAQ (SQL Server DBA Administration)**

🔷 **Query Store & Execution Plans**

1. **Q:** How do you handle query regressions after a SQL Server upgrade?
   A: Enable Query Store, compare pre/post-upgrade plans, and force stable plans.
2. **Q:** What if a query has highly variable runtimes due to parameters?
   A: Use OPTIMIZE FOR UNKNOWN, OPTION (RECOMPILE), or Query Store plan forcing.
3. **Q:** How do you detect queries suffering from parameter sniffing?
   A: Look at plan cache with sys.dm_exec_query_stats and compare runtime stats across executions.
4. **Q:** When would you apply plan guides?
   A: For legacy applications where query code cannot be modified.
5. **Q:** How does Query Store help with performance tuning?
   A: It captures historical query plans, runtimes, and allows regression detection + plan forcing.

🔷 **Advanced Indexing**

6. **Q:** When do you use Columnstore indexes?
   A: For analytic/reporting workloads with large aggregations, not for high-frequency OLTP updates.
7. **Q:** Can Columnstore be combined with OLTP indexes?
   A: Yes, via clustered columnstore with non-clustered rowstore indexes for hybrid workloads.
8. **Q:** What is index compression and when is it useful?
   A: Page/row compression reduces storage and I/O at the cost of CPU; useful for read-heavy workloads.
9. **Q:** How do filtered indexes improve performance?
   A: By targeting selective queries, reducing index size and maintenance overhead.
10. **Q:** Why align indexes with partitioning keys?
    A: For efficient partition elimination and partition switching during ETL.

🔷 **Partitioning & Large Tables**

11. **Q:** What's the benefit of partition switching?
    A: Instant load/unload of data without logging or locking overhead.
12. **Q:** How does partition elimination help query performance?
    A: It ensures only relevant partitions are scanned, reducing I/O.

13. **Q:** What is a sliding window partitioning strategy?
    A: Continuously drop oldest partition and add new one for time-series/ETL data.
14. **Q:** How do you archive partitions efficiently?
    A: Move older partitions to cheaper filegroups or external storage.
15. **Q:** Why might parallel queries underperform on partitioned tables?
    A: Misaligned partitions and indexes can cause skewed work distribution.

🔷 Parallelism & Workload Tuning

16. **Q:** What causes skewed parallelism?
    A: Uneven data distribution or poor statistics leading to one thread doing most work.
17. **Q:** How do you tune MAXDOP?
    A: Based on core count, workload type (OLTP vs DW), and Microsoft guidelines.
18. **Q:** Why adjust Cost Threshold for Parallelism?
    A: Default (5) is too low, causing small queries to parallelize unnecessarily.
19. **Q:** What's the difference between CXPACKET and CXCONSUMER waits?
    A: CXPACKET indicates imbalance, CXCONSUMER is benign (normal parallelism).
20. **Q:** How do you fix NUMA-related performance issues?
    A: Configure SQL Server affinity or balance connections per NUMA node.

🔷 In-Memory OLTP

21. **Q:** When do you use memory-optimized tables?
    A: For latch/lock contention or extreme OLTP workloads needing microsecond latency.
22. **Q:** What's the benefit of memory-optimized table variables?
    A: They avoid TempDB usage and support stats (unlike normal table variables).
23. **Q:** When should SCHEMA_ONLY tables be used?
    A: For staging/queue tables where persistence is not required.
24. **Q:** Why use natively compiled stored procedures?
    A: They reduce CPU cycles by compiling directly into machine code.
25. **Q:** What is the biggest limitation of In-Memory OLTP?
    A: High memory demand and limited support for certain features (triggers, FKs, etc.).

🔷 TempDB & Concurrency

26. **Q:** What causes metadata contention in TempDB?
    A: Heavy concurrent object creation; fix with memory-optimized TempDB metadata (SQL 2019+).
27. **Q:** Why add multiple TempDB files?
    A: To spread allocation contention across multiple PFS/GAM pages.
28. **Q:** How do you reduce spills to TempDB?
    A: Increase memory, tune queries, add indexes.
29. **Q:** What's the impact of Trace Flag 1118?
    A: Forces full extent allocations (pre-SQL 2016), reducing contention.
30. **Q:** Why dedicate storage for TempDB logs?
    A: To reduce contention with user database log writes.

🔷 Locking & Isolation

31. **Q:** How do you fix recurring deadlocks?
    A: Identify the victim query with XEvents, add indexes, reorder access paths.

32. **Q:** What's the advantage of RCSI?
    A: Eliminates reader-writer blocking by using row versions in TempDB.
33. **Q:** Why break long transactions into smaller ones?
    A: To reduce lock duration and blocking chains.
34. **Q:** How do you avoid lock escalation?
    A: Partitioning + ROWLOCK hints + smaller batch updates.
35. **Q:** What's a scenario for application-level queuing?
    A: Heavy writer contention where serialized access improves throughput.

🔷 I/O & Storage

36. **Q:** How do you reduce WRITELOG waits?
    A: Place logs on SSD/NVMe, optimize batch commits, avoid excessive autocommit.
37. **Q:** How do you detect I/O bottlenecks?
    A: Use sys.dm_io_virtual_file_stats and compare latencies to thresholds (<5 ms ideal).
38. **Q:** Why use instant file initialization?
    A: To eliminate zeroing overhead during file growth.
39. **Q:** What's the challenge with VLDB index maintenance?
    A: Long downtime; fix with partitioned rebuilds or online index rebuilds.
40. **Q:** How do you minimize checkpoint I/O spikes?
    A: Adjust recovery interval, stagger workloads, and optimize logging.

🔷 High Availability & Read Scaling

41. **Q:** How do you offload reporting from primary replicas?
    A: Route read workloads to AG readable secondaries.
42. **Q:** What causes query timeouts during AG failover?
    A: Lack of retry logic in applications.
43. **Q:** How do you reduce backup impact on primary?
    A: Configure backups on secondary replicas.
44. **Q:** How do you optimize analytics queries on AG?
    A: Use distributed AGs for scale-out.
45. **Q:** How do you address log transport bottlenecks?
    A: Enable compression, but monitor CPU overhead.

🔷 Cloud & Modern Features

46. **Q:** How do you optimize Azure SQL DB performance?
    A: Right-size DTUs/vCores, enable auto-tuning (indexes, plan forcing).
47. **Q:** What's unique about Managed Instance performance tuning?
    A: Same as on-prem, but constrained by tier (TempDB/IO sizing is service-tier based).
48. **Q:** Why use accelerated networking in Azure VM SQL?
    A: To reduce network latency for AG/log shipping.
49. **Q:** How do you optimize DR to blob storage?
    A: Compress/encrypt log backups and monitor blob latency.
50. **Q:** How does Synapse Serverless SQL pool optimization differ?
    A: Partition external tables, optimize file sizes, avoid small parquet files.

Top 50 – Query Store & Plan Management FAQ's (SQL Server DBA Administration)

Query Store Basics & Setup

1. **Q:** What is Query Store?
   A: A feature introduced in SQL Server 2016 that captures query runtime statistics, execution plans, and performance history.
2. **Q:** How do you enable Query Store?
   A: ALTER DATABASE [DBName] SET QUERY_STORE = ON;
3. **Q:** What are Query Store capture modes?
   A: ALL, AUTO, NONE. AUTO captures only expensive queries.
4. **Q:** How do you configure Query Store size limits?
   A: Via MAX_STORAGE_SIZE_MB and CLEANUP_POLICY settings.
5. **Q:** How do you view Query Store runtime stats?
   A: Using DMVs like sys.query_store_runtime_stats and sys.query_store_runtime_stats_interval.

Execution Plan Management

6. **Q:** What is a forced plan?
   A: A plan manually forced to be used by Query Store to prevent regressions.
7. **Q:** How do you force a plan?
   A: EXEC sp_query_store_force_plan @query_id, @plan_id;
8. **Q:** Can a forced plan be undone?
   A: Yes, using sp_query_store_unforce_plan.
9. **Q:** What's the difference between automatic plan correction and manual forcing?
   A: Automatic uses AUTO_PLAN_CORRECTION in SQL 2017+; manual forcing is DBA-controlled.
10. **Q:** How do you identify plan regressions?
    A: Compare runtime stats over intervals via Query Store DMVs.

Parameter Sniffing & Query Store

11. **Q:** How does Query Store help with parameter sniffing?
    A: It shows plan performance variance across parameter values; forces stable plans if needed.
12. **Q:** Can Query Store detect bad parameter sniffing automatically?
    A: SQL 2017+ can detect and optionally fix via Automatic Plan Correction.
13. **Q:** How do you force a generic plan to avoid parameter sniffing?
    A: Use OPTION (OPTIMIZE FOR UNKNOWN) or Query Store plan forcing.
14. **Q:** Can Query Store track per-parameter performance?
    A: It tracks aggregate performance; SQL 2022+ introduces more detailed stats.
15. **Q:** How to validate if plan forcing improved parameter sniffing issues?
    A: Compare query runtime, logical reads, CPU usage pre- and post-forcing.

Plan History & Analysis

16. **Q:** How long does Query Store retain data?
    A: Configurable via CLEANUP_POLICY (e.g., 30 days default).
17. **Q:** How do you purge Query Store data?
    A: ALTER DATABASE [DBName] SET QUERY_STORE CLEAR;
18. **Q:** How can you identify the top resource-consuming queries?
    A: Use sys.query_store_query + sys.query_store_runtime_stats sorted by avg CPU/IO.
19. **Q:** How can you track historical plan changes?
    A: Join sys.query_store_plan with sys.query_store_query and sys.query_store_runtime_stats.

20. **Q:** What is the difference between last_execution_time and runtime stats interval?
    A: last_execution_time shows most recent execution; runtime stats interval aggregates multiple executions.

🔷 Troubleshooting & Performance Scenarios

21. **Q:** Query suddenly slowed down after upgrade → what to check?
    A: Compare pre/post Query Store plan history; consider plan forcing.
22. **Q:** Execution plan regression detected → solution?
    A: Force previous optimal plan, update statistics, consider index tuning.
23. **Q:** Query uses high CPU only for certain parameters → solution?
    A: Force plan suitable for common parameters, OPTION (RECOMPILE) for edge cases.
24. **Q:** Query execution shows large variance → Query Store strategy?
    A: Use runtime stats intervals to analyze variance; consider automatic plan correction.
25. **Q:** How to troubleshoot batch ETL queries with Query Store?
    A: Track plan performance over intervals; identify spilling or TempDB issues.

🔷 DMVs & Query Store Reporting

26. **Q:** Key DMVs for Query Store analysis?
    A: sys.query_store_query, sys.query_store_plan, sys.query_store_runtime_stats, sys.query_store_runtime_stats_interval.
27. **Q:** How to identify queries with multiple plans?
    A: sys.query_store_query_plan grouped by query_id.
28. **Q:** How to find plans with highest duration?
    A: Sort avg_duration in sys.query_store_runtime_stats.
29. **Q:** How to get execution count per plan?
    A: execution_count column in sys.query_store_runtime_stats.
30. **Q:** How to correlate runtime stats with plan IDs?
    A: Join sys.query_store_plan.plan_id with sys.query_store_runtime_stats.plan_id.

🔷 Automatic Plan Correction (SQL 2017+)

31. **Q:** What is Automatic Plan Correction?
    A: Feature that automatically detects regressions and forces previous optimal plans.
32. **Q:** How to enable it?
    A: ALTER DATABASE [DBName] SET AUTOMATIC_TUNING = FORCE_LAST_GOOD_PLAN;
33. **Q:** How does it detect regressions?
    A: Compares plan runtime stats; if new plan is slower than previous baseline, it triggers correction.
34. **Q:** Can automatic tuning interfere with manual plan forcing?
    A: Manual plan forcing takes precedence; automatic tuning respects forced plans.
35. **Q:** How to monitor automatic plan corrections?
    A: sys.database_automatic_tuning_options and Query Store plan stats.

🔷 Real-World Scenarios

36. **Q:** Query plan changes after index rebuild → what to check?
    A: Check Query Store for new plan performance; consider forcing if regression occurs.
37. **Q:** High variance queries on AG readable secondary → solution?
    A: Monitor Query Store on primary for plan performance; replicate plan forcing as needed.
38. **Q:** Query Store data growth → mitigation?
    A: Limit capture mode, reduce retention, pre-size storage.

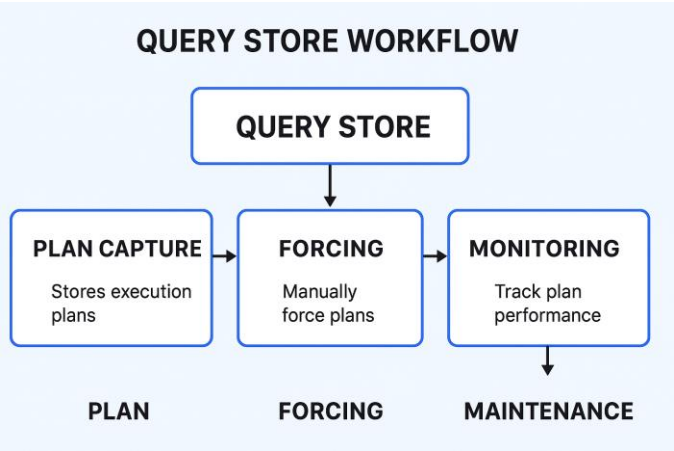
39. **Q:** ETL batch queries showing different plans each run → solution?
    A: Force stable plan for high-volume queries; use RECOMPILE selectively.
40. **Q:** Multi-parameter query has poor plan → what Query Store insight helps?
    A: Compare plan performance per parameter range; choose plan for most common path.

🔷 Plan Forcing Best Practices

41. **Q:** When should you force a plan?
    A: Only after confirming that previous plan consistently outperforms new plan.
42. **Q:** Can forced plans be harmful?
    A: Yes, if underlying data changes significantly or indexes are modified.
43. **Q:** How often should Query Store data be reviewed?
    A: Weekly or after major deployments, schema changes, or version upgrades.
44. **Q:** How to avoid excessive plan forcing?
    A: Use automatic plan correction when possible; force only critical queries.
45. **Q:** How to force plans for multiple queries in batch?
    A: Script sp_query_store_force_plan for all query_ids needing plan stability.

🔷 Maintenance & Monitoring

46. **Q:** How to clear Query Store without downtime?
    A: ALTER DATABASE [DBName] SET QUERY_STORE CLEAR; — does not affect current connections.
47. **Q:** How to check Query Store health?
    A: Monitor capture rate, storage usage, and runtime stats completeness.
48. **Q:** How to manage storage growth?
    A: Set MAX_STORAGE_SIZE_MB, monitor capture intervals, enable AUTO cleanup.
49. **Q:** What's a common pitfall when enabling Query Store?
    A: Enabling on very busy systems without sizing storage properly can cause performance impact.
50. **Q:** How to integrate Query Store analysis into daily monitoring?
    A: Combine with DMVs, custom reports, and alerting for regressions or performance anomalies.



🔍 Top 50 Extended Events & Monitoring Scenarios FAQs – SQL Server DBA

🔷 Basics & Setup

1. **What are Extended Events (XEvents) in SQL Server?**
   A lightweight performance monitoring system built into SQL Server that allows users to collect data for troubleshooting and analysis.
2. **How are XEvents different from SQL Trace / Profiler?**
   XEvents are more performant, offer greater flexibility, are asynchronous, and are not deprecated (unlike Profiler/Trace).
3. **How do I create a basic XEvent session?**
   Use T-SQL or SSMS GUI:
4. CREATE EVENT SESSION MySession
5. ON SERVER
6. ADD EVENT sqlserver.sql_batch_completed
7. ADD TARGET package0.event_file (SET filename='C:\XEvents\MySession.xel');
8. **Where is XEvent data stored?**
   - In memory (ring buffer)
   - Event file (.xel)
   - Live data stream (SSMS)
9. **What is the difference between ring_buffer and event_file?**
   ring_buffer stores data in memory, ideal for short-lived sessions; event_file stores data on disk, best for longer sessions and persistence.

🔷 Monitoring and Performance Scenarios

6. **How to monitor slow-running queries with XEvents?**
   Use sql_batch_completed or rpc_completed events, filter on duration.
7. **How to capture deadlocks with Extended Events?**
   Capture the xml_deadlock_report event and write it to a file.
8. **How can I track blocking sessions?**
   Use blocked_process_report (requires blocked process threshold setting in sp_configure).
9. **How to track failed logins or login events?**
   Use sqlserver.login, sqlserver.logout, and sqlserver.failed_login.
10. **How do I capture high CPU queries?**
    Use sql_batch_completed and filter based on cpu_time.

🔷 Security & Audit Use Cases

11. **Can I audit schema changes via XEvents?**
    Yes, use object_altered, object_created, object_deleted.
12. **How to capture permission errors or denied access?**
    Use security_error, error_reported, or login_failed.
13. **How to monitor user activity (DDL/DML)?**
    Capture sql_statement_completed, sql_batch_completed, and filter by username.

🔷 Advanced Tuning & Troubleshooting

14. **How to trace tempdb contention via XEvents?**
    Use events like wait_info, wait_completed, filtered on PAGELATCH_* waits.
15. **Can I detect parameter sniffing problems?**
    Monitor query_post_execution_showplan to compare execution plans for the same query.

16. **How to capture execution plans?**
    Use query_post_execution_showplan or query_pre_execution_showplan.
17. **How do I track recompilations?**
    Capture sqlserver.sql_statement_recompile.
18. **How can I detect memory grants or memory pressure?**
    Use query_memory_grant, memory_grant_updated_by_feedback, and resource_monitor_ring_buffer_recorded.
19. **Can I detect tempdb spills?**
    Yes, via sort_warning or hash_warning.
20. **How to monitor long compile times?**
    Use sql_statement_recompile, sql_batch_starting, sql_batch_completed.

🔷 Target Management

21. **What are the different targets in Extended Events?**

- event_file
- ring_buffer
- event_counter
- histogram
- asynchronous_file_target

22. **How to read .xel files?**
    Use SSMS GUI, or sys.fn_xe_file_target_read_file().
23. **How to automatically delete old XEvent files?**
    Set max_file_size and max_rollover_files.
24. **Can I store XEvent output to a table?**
    Not directly, but you can import .xel file contents into a table using T-SQL.

🔷 Real-World Scenarios

25. **Monitor queries causing excessive IO**
    Use sql_batch_completed and check logical_reads, physical_reads.
26. **Monitor queries with frequent recompiles**
    Use sql_statement_recompile and group by query_hash.
27. **Capture data for a specific application user**
    Filter using client_app_name or username.
28. **Track index usage or missing index hints**
    Use query_post_execution_showplan and analyze plan XML for missing index warnings.
29. **Detect implicit conversions**
    Use plan_affecting_convert event.
30. **Monitor wait stats in real-time**
    Use wait_info, wait_info_external.

🔷 XEvent Management & Best Practices

31. **How to find existing XEvent sessions?**
    Query sys.server_event_sessions.
32. **How to script out an XEvent session?**
    Right-click the session in SSMS > Script Session as > CREATE.
33. **Can I pause and resume sessions?**
    Yes, using ALTER EVENT SESSION ... STATE = STOP / START.

34. **How to check if an XEvent session is active?**
    Check sys.dm_xe_sessions.
35. **Do XEvents have performance overhead?**
    Minimal, especially when using asynchronous targets like event_file.
36. **How to optimize filtering in XEvents?**
    Use predicates (WHERE) in session definition to reduce unnecessary data collection.
37. **How to correlate XEvents with other DMVs?**
    Use session_id or plan_handle to join with DMVs like sys.dm_exec_requests.

🔷 Custom Scenarios

38. **Monitor queries with high row count returned**
    Use sql_batch_completed, check row_count.
39. **Track long-running transactions**
    Combine sql_transaction_starting, sql_transaction_ended with duration checks.
40. **Monitor auto-grow events on database files**
    Use database_file_size_change.
41. **Detect long waits on specific resources**
    Use wait_info, filter by wait_type.
42. **Trace latch contention**
    Use latch_suspend_begin and latch_suspend_end.

🔷 Diagnostics & Debugging

43. **How to debug query timeouts?**
    Use attention, rpc_completed, and sql_batch_completed.
44. **Capture sp_server_diagnostics data**
    Use the system health session or enable sp_server_diagnostics_component_result.
45. **How to detect plan cache bloat?**
    Monitor query_cache_removal_statistics.
46. **Can XEvents replace all SQL Profiler use cases?**
    Mostly yes. Some legacy events are not available, but for modern usage, XEvents is preferred.

🔷 Built-in Sessions

47. **What is the 'system_health' session?**
    A built-in session that captures key diagnostic data automatically.
48. **Can I modify built-in XEvent sessions?**
    No, but you can stop and start them. To customize, create a new session.
49. **How to monitor availability group events?**
    Use availability_replica_*, availability_group_* events.
50. **How to monitor backup/restore operations?**
    Use backup_restore_progress_trace, backup_completed, and restore_completed.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧠 Top 50 Deadlocks & Concurrency Scenarios FAQs — SQL Server DBA

🔷 1–10: Fundamentals of Deadlocks and Concurrency

1. **What is a deadlock in SQL Server?**
   A deadlock occurs when two or more sessions block each other by holding a lock on a resource the other session needs.
2. **What is concurrency in SQL Server?**
   Concurrency refers to multiple processes accessing and modifying the same data at the same time.
3. **What causes deadlocks?**
   - Conflicting resource access order
   - Long-running transactions
   - Inefficient indexing
   - Escalated locks
4. **What is a deadlock victim?**
   SQL Server chooses one of the processes as a victim (usually the one with the lowest cost) and terminates it with error 1205.
5. **How does SQL Server detect deadlocks?**
   SQL Server uses a lock monitor thread that checks for deadlocks every 5 seconds.
6. **What are the different types of locks in SQL Server?**
   - Shared (S), Exclusive (X), Update (U), Intent (IS, IX), Schema, Key-Range, etc.
7. **What is lock escalation?**
   SQL Server escalates from row/page locks to table locks to conserve memory.
8. **What is transaction isolation level?**
   Defines how/when data changes made by one transaction become visible to others (e.g., Read Committed, Serializable).
9. **Which isolation levels are more prone to deadlocks?**
   - Serializable and Repeatable Read can increase contention.
   - Read Committed Snapshot (RCSI) often reduces deadlocks.
10. **How to find out if deadlocks are occurring in SQL Server?**
    Look for error 1205 in the error log, use Extended Events (xml_deadlock_report), or use the system_health session.

🔷 11–20: Deadlock Detection and Analysis

11. **How can I capture a deadlock graph?**
    Use Extended Events (xml_deadlock_report) or the default system_health session.
12. **Where is the system_health session located?**
    Under Management > Extended Events > Sessions > system_health.
13. **How to read a deadlock graph?**
    Analyze the XML or use SSMS graphical view: shows processes, resources, and victim.
14. **What tools can help visualize deadlocks?**
    - SSMS
    - SQL Sentry Plan Explorer
    - XEvent Viewer
    - Deadlock Graph Viewer
15. **How to query the system_health XEvent for deadlocks?**
16. SELECT CAST(event_data AS XML) AS deadlock_graph
17. FROM sys.fn_xe_file_target_read_file(
18. 'system_health*.xel', NULL, NULL, NULL)
19. WHERE event_data LIKE '%deadlock%';
20. **What are common patterns in deadlocks?**

- Reader vs. Writer
- Writer vs. Writer
- Key lookup + clustered update
- Serial deadlocks

21. **Can deadlocks occur without user transactions?**
    Yes — internal processes (e.g., statistics updates, system threads) can also deadlock.
22. **What is a conversion deadlock?**
    Occurs when two sessions need to convert their locks and block each other.
23. **How to use SQL Profiler to trace deadlocks?**
    Include Deadlock Graph event class in your trace (note: deprecated in favor of XEvents).
24. **What are Key-Range locks, and how do they affect deadlocks?**
    Used in Serializable isolation to prevent phantom reads, increasing deadlock risk.

🔷 21–30: Prevention & Best Practices

21. **How to prevent deadlocks in SQL Server?**
    - Keep transactions short
    - Access objects in a consistent order
    - Use proper indexing
    - Use RCSI
    - Avoid unnecessary locking hints
22. **How to set transaction isolation level to avoid deadlocks?**
    Use SET TRANSACTION ISOLATION LEVEL READ COMMITTED or SNAPSHOT.
23. **Can NOLOCK hints prevent deadlocks?**
    They reduce locking but introduce dirty reads — not always safe or recommended.
24. **What is Snapshot Isolation?**
    A row-versioning-based isolation level that avoids shared locks on reads.
25. **How to enable Read Committed Snapshot Isolation (RCSI)?**
26. ALTER DATABASE [YourDB]
27. SET READ_COMMITTED_SNAPSHOT ON;
28. **Does indexing reduce deadlocks?**
    Yes. Proper indexes reduce scan time, lock duration, and key conflicts.
29. **Can partitioning help reduce deadlocks?**
    Yes, by reducing contention on hot tables or indexes.
30. **How to avoid deadlocks in stored procedures?**
    - Use consistent object access order
    - Avoid user interaction in transactions
    - Reduce transaction scope
31. **How to use TRY...CATCH to handle deadlocks?**
    Catch error 1205 and retry the transaction:
    BEGIN TRY

```
BEGIN TRANSACTION
    -- work
COMMIT
END TRY
BEGIN CATCH
IF ERROR_NUMBER() = 1205
    BEGIN
```

```
        -- retry logic
END
        ELSE
        THROW;
END CATCH
```

32. **How many retries are advisable in deadlock retry logic?**
    Usually 3 retries with exponential backoff is safe.

🔷 31–40: Real-World Deadlock Scenarios

31. **Deadlock during index rebuild and select queries**
    Readers block rebuild process if not using ONLINE=ON; rebuild can escalate locks.
32. **Deadlock between two updates on different tables**
    Often due to update order mismatch or foreign key constraints.
33. **Deadlock between SELECT and INSERT**
    Caused by SELECT holding shared locks and INSERT needing exclusive locks.
34. **Deadlocks involving triggers**
    Triggers running implicit transactions can hold locks longer.
35. **Deadlocks caused by cascading updates or deletes**
    FK relationships with cascading can lead to nested transactions and lock chains.
36. **Deadlock with SSIS packages**
    Parallel data flow tasks accessing the same destination can deadlock.
37. **Deadlocks during index maintenance**
    Offline index rebuilds lock tables and block user queries.
38. **Deadlocks due to lack of WHERE clause in UPDATE/DELETE**
    Full table scans cause wider locks and more contention.
39. **Deadlock between reporting and OLTP workloads**
    Long-running reporting queries holding shared locks block writers.
40. **Deadlocks involving temp tables**
    Temp tables can still acquire locks depending on isolation level and query patterns.

🔷 41–50: Monitoring, Tuning & Configuration

41. **How to monitor lock waits in real time?**
    Query sys.dm_os_waiting_tasks and sys.dm_tran_locks.
42. **How to get current blocking session info?**
43. SELECT * FROM sys.dm_exec_requests WHERE blocking_session_id <> 0;
44. **What is the blocked process threshold setting?**
    Enables detection of blocked processes via XEvents:
45. EXEC sp_configure 'show advanced options', 1;
46. RECONFIGURE;
47. EXEC sp_configure 'blocked process threshold', 10;
48. RECONFIGURE;
49. **Can deadlocks be resolved automatically?**
    SQL Server always resolves deadlocks by choosing a victim; you can handle it gracefully in code.
50. **How to identify frequently deadlocking queries?**
    Use Extended Events + query hash + execution plan analysis.

51. **How to simulate a deadlock?**
    Open two SSMS windows and run conflicting transactions with BEGIN TRAN.
52. **How to check transaction isolation level of a session?**
    Use DBCC USEROPTIONS or query sys.dm_exec_sessions.
53. **Are deadlocks logged in SQL Server Error Log?**
    Only if trace flag 1222 or 1204 is enabled.
54. **How to enable trace flag 1222 for deadlock logging?**
55. DBCC TRACEON (1222, -1);
56. **Can I automate deadlock analysis?**
    Yes. Use XEvent session + job to parse .xel files and store them for later analysis.

⚙ Top 50 Parallelism & MAXDOP Scenarios FAQs – SQL Server DBA

🔷 1–10: Basics of Parallelism and MAXDOP

1. **What is parallelism in SQL Server?**
   It's SQL Server's ability to divide query execution into multiple threads across CPUs to improve performance.
2. **What is MAXDOP?**
   MAXDOP (Maximum Degree of Parallelism) controls the number of processors SQL Server uses for a single query or index operation.
3. **How does SQL Server decide to use parallelism?**
   Based on estimated query cost and server settings like cost threshold for parallelism.
4. **What is the default MAXDOP setting in SQL Server?**
   By default, it's 0, meaning SQL Server can use all available CPUs.
5. **What is the cost threshold for parallelism?**
   The minimum estimated cost a query must have to be considered for parallelism. Default is 5 (which is often too low).
6. **How to check the current MAXDOP setting?**
7. SELECT value_in_use FROM sys.configurations WHERE name = 'max degree of parallelism';
8. **How to change MAXDOP globally?**
9. EXEC sp_configure 'max degree of parallelism', 4;
10. RECONFIGURE;
11. **Can MAXDOP be set at the database level?**
    Yes, starting from SQL Server 2016 (13.x):
12. ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;
13. **Can MAXDOP be set per query?**
    Yes, using the OPTION (MAXDOP N) query hint.
14. **How is parallelism related to CPU usage?**
    Parallel queries can cause high CPU usage if not controlled, leading to resource contention.

🔷 11–20: Query Performance & Execution

11. **When should I consider using MAXDOP = 1?**
    For OLTP workloads, to avoid excessive CPU usage or query skewing from parallelism.
12. **What are the signs of excessive parallelism?**
    - High CPU usage
    - CXPACKET waits
    - Queries using all cores inefficiently
    - Query plan shows parallelism operators

13. **What is a CXPACKET wait type?**
    Indicates a parallel query has threads waiting to synchronize. Not always bad, but worth analyzing.
14. **What is CXCONSUMER wait type?**
    Represents idle parallel worker threads waiting to consume rows. Usually benign.
15. **Can a query run slower with parallelism?**
    Yes. Poorly distributed workloads across threads can cause skew and overhead.
16. **How can I see if a query used parallelism?**
    Check execution plan for "Parallelism" operators or Actual Number of Rows per Execution.
17. **How to find all currently running parallel queries?**
18. SELECT * FROM sys.dm_exec_requests WHERE dop > 1;
19. **How to see the degree of parallelism in a query plan?**
    In the actual execution plan, look for Estimated/Actual Degree of Parallelism.
20. **What is query plan skew?**
    Occurs when one parallel thread processes significantly more rows than others.
21. **What causes skew in parallelism?**
    - Data skew (uneven distribution)
    - Poor stats
    - Hash joins with uneven buckets

🔷 21–30: Tuning & Best Practices

21. **What's a recommended starting MAXDOP value?**
    - OLTP: 1–4
    - DW/OLAP: 8–MAX
    - Follow Microsoft guidelines based on core layout and NUMA.
22. **How to find the ideal cost threshold for parallelism?**
    Monitor your query costs; many environments benefit from increasing it to 25–50 or higher.
23. **How to analyze parallel plans vs serial plans?**
    Compare estimated/actual execution plans using SSMS or Query Store.
24. **Should I disable parallelism for small queries?**
    Yes — small queries don't benefit from parallelism and just add overhead.
25. **How to find queries that overuse parallelism?**
    Use Query Store, and filter by high DOP or high CPU consumption.
26. **How does tempdb affect parallelism?**
    Parallel plans may use tempdb heavily (e.g., spills), especially in hash joins.
27. **How to reduce tempdb contention from parallel plans?**
    Optimize queries and indexes to avoid spills; ensure tempdb is properly configured (multiple files, TF 1117/1118, etc.).
28. **Can indexes cause unwanted parallel plans?**
    Yes — missing or inefficient indexes may lead SQL Server to use parallelism unnecessarily.
29. **Should MAXDOP be limited on a multi-instance server?**
    Yes — especially if multiple SQL Server instances share CPU resources.
30. **Does MAXDOP apply to index maintenance?**
    Yes. Index rebuilds can run in parallel unless MAXDOP=1 is specified or set in the task.

🔷 31–40: Configuration, Hardware, and NUMA

31. **What is the role of NUMA in parallelism?**
    SQL Server aligns threads to NUMA nodes for performance. Misalignment can cause performance drops.

32. **How do I view my server's NUMA configuration?**
    Use:
33. SELECT node_id, cpu_count FROM sys.dm_os_nodes WHERE memory_node_id < 64;
34. **How to calculate recommended MAXDOP for NUMA systems?**
    Follow Microsoft's updated guidance (2023):
    - ≤8 CPUs: MAXDOP = 0 or #logical CPUs
    - 8 CPUs: MAXDOP = #logical CPUs per NUMA node (up to 16)
35. **Should MAXDOP match physical cores or logical CPUs?**
    Use logical CPUs (includes hyperthreading) for MAXDOP settings.
36. **Is it better to set MAXDOP at query, database, or server level?**
    Query-level provides granular control. Use DB or server level for defaults.
37. **How to enforce MAXDOP for third-party applications?**
    Use Resource Governor or a plan guide to limit MAXDOP.
38. **Does MAXDOP affect scalar UDFs or TVFs?**
    Yes — UDFs used in WHERE clauses may force serial execution in older versions.
39. **Does parallelism occur inside triggers?**
    Generally no — triggers often run serially due to transaction scope.
40. **How does memory pressure affect parallelism?**
    SQL Server may avoid parallel plans under memory pressure to reduce worker/thread count.
41. **What is "Parallel Plan Reason" in execution plan?**
    SQL Server 2022+ shows why a query was (or was not) parallelized (e.g., cost threshold, MAXDOP limits).

🔷 41–50: Monitoring, Troubleshooting & Real-World Scenarios

41. **How to monitor parallel plan usage over time?**
    Use Query Store or Extended Events (e.g., query_post_execution_showplan).
42. **Can Resource Governor control MAXDOP?**
    Yes. You can create workload groups with specific MAXDOP limits.
43. **How to detect parallel plan regressions?**
    Use Query Store to compare plan versions, DOPs, and performance metrics.
44. **Why is MAXDOP = 1 sometimes faster?**
    Less overhead, especially for small OLTP queries with low row counts.
45. **Can MAXDOP cause CPU starvation?**
    Yes — high DOP queries can saturate all CPU threads, starving others.
46. **How to limit MAXDOP during index rebuilds?**
    Use:
47. ALTER INDEX REBUILD ... WITH (MAXDOP = 1);
48. **Should I use MAXDOP = 0 in production?**
    Rarely. It allows full CPU usage, which may harm concurrency and stability.
49. **What's the impact of MAXDOP in In-Memory OLTP?**
    In-memory tables are designed for high concurrency; avoid high MAXDOP for transactions accessing them.
50. **How does query parallelism affect plan caching?**
    Queries with different MAXDOP hints may not reuse cached plans efficiently.
51. **Can SQL Server parallelize DML statements (INSERT/UPDATE/DELETE)?**
    Yes, but not always. It depends on the query structure, indexes, and estimated cost.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

📘 Top 50 Indexing Advanced Scenarios FAQs — SQL Server DBA

🔷 1–10: Core Index Concepts Revisited

1. **What is the difference between clustered and non-clustered indexes?**
   - **Clustered Index**: Sorts and stores the data rows in the table based on the key. One per table.
   - **Non-Clustered Index**: Separate structure with pointers (RID or clustering key) to the actual data.
2. **Can a table have multiple clustered indexes?**
   No. Only one clustered index per table is allowed.
3. **What is an included column in a non-clustered index?**
   Non-key column(s) stored at the leaf level to support covering queries without becoming part of the index key.
4. **What is index fragmentation and why does it matter?**
   - **Logical fragmentation**: Non-contiguous pages.
   - Affects performance of range scans and can increase IO.
5. **What is the fill factor and how does it affect indexing?**
   Determines the percentage of space filled per page during index creation/rebuild. Lower fill factor reduces page splits.
6. **When should I use filtered indexes?**
   For selective queries with known WHERE conditions. Example: IsDeleted = 0.
7. **How are unique indexes different from constraints?**
   Both enforce uniqueness, but **constraints** are for data integrity (metadata), and **indexes** are for performance.
8. **What is the impact of indexing on write operations?**
   - Inserts, updates, and deletes must maintain index structures, which can add overhead.
9. **How does SQL Server choose an index?**
   Based on the **query optimizer's cost model**, statistics, selectivity, and index coverage.
10. **What is index selectivity?**
    The **ratio of unique values** to total rows. High selectivity indexes are more useful.

🔷 11–20: Index Types and Use Cases

11. **What is a covering index?**
    An index that includes all columns used by a query, eliminating the need to access the base table (RID/lookup avoided).
12. **When should I use a composite (multi-column) index?**
    When queries filter or sort on multiple columns in a predictable order.
13. **How is the order of columns in an index important?**
    The index is **only usable for seeks** if the **leading column(s)** are part of the WHERE or JOIN condition.
14. **What are spatial and XML indexes?**

- **Spatial**: For geometry/geography data types.
- **XML**: Improve performance on XPath/XQuery operations.

15. **What are full-text indexes?**
    Used for efficient search on character-based data with linguistic support (e.g., stemming, inflectional forms).
16. **What are columnstore indexes?**

- **Compressed, column-oriented** storage for OLAP/analytics.
- Great for scanning and aggregation.

17. **Can I combine columnstore and traditional indexes?**
    Yes. For example, on hybrid OLTP-OLAP systems using **clustered columnstore** with filtered rowstore indexes.
18. **What is an index on a computed column?**
    You can index deterministic and persisted computed columns to improve performance.
19. **Can I use indexes on temp tables?**
    Yes. Create explicit indexes or rely on automatically created clustered index on primary key/temp table structure.

20. **What is the impact of indexing on query plans?**
    Indexes can enable **seeks**, **covering**, **merges**, or **hash joins**, and reduce **sort**/scan operations.

🔷 21–30: Index Maintenance & Monitoring

21. **How do I check index fragmentation?**
    Use:

```
SELECT * FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED');
```

22. **When should I rebuild vs. reorganize an index?**

- Rebuild: >30% fragmentation
- Reorganize: 5–30% fragmentation

    Use a script to automate thresholds.

23. **How to schedule index maintenance efficiently?**
    Use jobs with logic based on fragmentation %, usage stats, and off-peak windows.
24. **Do filtered indexes need special maintenance?**
    Yes — they can become stale if data distribution changes outside the filter condition.
25. **How do I detect unused indexes?**
    Use:

```
SELECT * FROM sys.dm_db_index_usage_stats WHERE user_seeks = 0 AND user_scans = 0;
```

26. **How to detect missing indexes?**
    Query:

```
SELECT * FROM sys.dm_db_missing_index_details;
```

27. **Can missing index suggestions be trusted?**
    Not always. They are **query-specific**, don't consider existing indexes, and may be **redundant**.
28. **How to monitor index usage trends over time?**
    Log values from sys.dm_db_index_usage_stats periodically and analyze deltas.
29. **Does rebuilding an index update statistics?**
    Yes — **rebuild updates stats with full scan**, **reorganize does not**.
30. **Should I rebuild indexes on heaps?**
    You can't — but you can use REBUILD to convert a heap into a clustered index.

🔷 31–40: Advanced Index Design Scenarios

31. **How many non-clustered indexes should a table have?**
    Depends on workload — each comes at a cost. Avoid indexing every column.
32. **Can indexes hurt performance?**
    Yes. Too many indexes can slow writes and confuse the optimizer.
33. **What's the impact of wide indexes?**
    More storage, higher maintenance cost, but good for covering wide queries.
34. **When to consider index compression?**
    For **large tables** with repetitive values. Use ROW or PAGE compression to reduce IO.
35. **Can indexes help with OR conditions?**
    Sometimes. Consider **filtered indexes** or **index union strategies**.
36. **What are indexed views and when to use them?**

- Materialized views stored on disk.
- Useful for aggregations or expensive joins.

37. **Can I index CTEs or subqueries?**
    Not directly — but you can refactor into views or temp tables with indexes.

38. **How to index a table with frequent inserts?**
    Minimize indexes to avoid overhead. Use **heap or single clustered index**, consider partitioning.
39. **How to index for range scans (BETWEEN, >, <)?**
    Use clustered or covering indexes with the range column as the **leading key**.
40. **What is the strategy for indexing datetime columns?**
    Index as the **first key column** if queries filter by date range; consider **partitioning** large historical data.

🔷 41–50: Real-World Scenarios & Troubleshooting

41. **Why is SQL Server not using my index?**
    Possible reasons:
- Outdated stats
- Low selectivity
- Index not covering
- Incorrect key column order

42. **How to force index usage?**
    Use query hint: WITH (INDEX(index_name)) — but only after understanding why the optimizer skipped it.
43. **Why is an index seek turning into a scan?**
    Could be due to **non-sargable predicates**, bad stats, or data skew.
44. **What is sargability?**
    Query condition can use indexes efficiently (e.g., WHERE column = @val vs. WHERE ISNULL(column, 0) = 1).
45. **Why is SQL using a key lookup?**
    The non-clustered index doesn't include all columns needed. Solution: **covering index** or **INCLUDE** clause.
46. **What are "tipping point" queries in indexing?**
    Small vs. large result sets — optimizer may switch from seek to scan depending on parameter values and estimates.
47. **How to handle hot spots in index contention?**
- Use **sequential vs. random keys** wisely
- Consider **partitioning** or **GUIDs**

48. **How does indexing affect concurrency?**
    Indexes can reduce locking contention by limiting the rows/pages accessed — but can also increase lock duration on updates.
49. **What is an ascending key problem?**
    When inserting into the end of a B-tree (e.g., identity columns), can cause **latch contention**. Consider **sequential GUIDs** or **partitioning**.
50. **How to test the effectiveness of an index?**
    Compare query performance and execution plans **before/after** index creation using SET STATISTICS IO/TIME ON.

🛡 **Always On & High Availability Performance Scenarios FAQs — SQL Server DBA**

These are grouped by theme: architecture & basics; performance & configuration; monitoring & metrics; troubleshooting & edge scenarios; advanced & scaling.

1–10: Architecture & Basics

1. **What is Always On in SQL Server?**
   "Always On" refers to features for high availability and disaster recovery—including Availability Groups (AG) and Failover Cluster Instances (FCI).
2. **What's the difference between AG and FCI?**
   - FCI operates at the instance level; shared storage required; failover instance moves entire instance.
   - AG is database-level; replicas have independent storage; supports readable secondaries, more flexibility.
3. **What are the availability modes?**
   Synchronous-commit vs asynchronous-commit. Synchronous gives zero or minimal data loss but adds latency; asynchronous allows lag but better performance for geographically distant replicas.
4. **What is a readable secondary, and when to use it?**
   Secondary replicas configured as readable (for read-only workloads, reporting, backups) to offload work from primary. Helps performance if properly deployed.
5. **What is failover mode: automatic vs manual?**
   Automatic failover triggers without human intervention under predefined conditions (synchronous commit + quorum + witness etc.), manual is initiated by DBA/ops.
6. **What is quorum and why is it important?**
   Windows Server Failover Cluster (WSFC) quorum: ensures cluster has enough voting nodes to avoid split-brain. Faulty quorum can delay or prevent failover.
7. **What's a listener, and why is it important?**
   A virtual network name (and IP) that clients use; AG listener handles routing connections to the current primary (or read-only secondary). Ensures applications can transparently connect.
8. **What is failover cluster instance (FCI) storage needs?**
   Shared storage or storage replicated at block level to support active-passive instance. Storage performance is critical, as primary and failover both depend on it.
9. **What licensing/licensing edition constraints are there?**
   Depending on SQL Server edition, features vary: e.g., in earlier versions, some AG capabilities were enterprise-only. Also, readable secondaries, backup on secondary etc., licensing implications.
10. **What network considerations are needed?**
    Bandwidth, latency, consistency; network reliability; possibly multiple NICs; for cross datacenter AGs, WAN latency matters especially in sync mode.

11–20: Performance & Configuration

11. **What's the performance overhead of synchronous replicas?**
    Primary waits for acknowledgement from secondary for transaction log hardening. If secondary is slow (disk, network, redo), that adds latency to commit on primary. (Microsoft Learn)
12. **How many AG databases should be in a single AG?**
    There is a scalability limit. As the number of databases increases, resource usage (CPU, memory, log send, redo) rises, and failover time can increase significantly. E.g., experiments show having many AG databases slows down DB-level failover. (Microsoft Tech Community)
13. **How do I calculate / monitor RTO & RPO?**
    Use metrics such as log_send_queue_size, redo_queue_size, redo_rate, last_commit_time etc. The Always On performance docs lay out how to compute these. (Microsoft Learn)

14. **How to size and tune storage for AGs?**
    Fast I/O for transaction logs (primary + secondary), good write and read latency; redo performance on secondary; consider disk throughput as well as latency under load.
15. **What is the impact of network latency on AG RPO / sync performance?**
    Higher network latency increases delays during sync, especially for synchronous commit. It can also contribute to log send queue buildup and longer redo times.
16. **How to optimize redo performance on secondary replicas?**
    Use fast storage, reduce workload on secondary (e.g. limit other workloads besides redo/read-only), ensure sufficient CPU, avoid blocking IO, possibly using multiple redo threads if version supports.
17. **What is the log send queue, and when does it become a bottleneck?**
    It's the amount of log data waiting to be sent to secondary. If sending is slower than log generation, it builds up; this delays synchronization and increases RPO. Monitor its size and send rate. (Microsoft Learn)
18. **How to minimize performance impact on primary when AG is under heavy load?**
    Offload read-only workloads to readable secondaries; ensure that networks, disk, CPU are not saturated; possibly adjust commit mode (sync vs async) based on workload and latency; ensure proper listener configuration; avoid blocking operations.
19. **What settings control AG performance?**

- Synchronous vs asynchronous commit
- Automatic failover detection thresholds
- Max number of readable secondaries
- Seeding and reseeding strategies
- Backup preferences (primary vs secondary)

20. **How do automatic seeding and data movement impact performance?**
    When new replicas are added or failover occurs, data needs to be seeded—this uses resources (network, disk IO). If done during peak hours, can degrade performance. It's better to schedule during low use or use fast networks.

21–30: Monitoring & Metrics

21. **Which DMVs/counters are most useful for Always On performance?**

- sys.dm_hadr_database_replica_states (log send / redo lag, role, synchronization health)
- Performance Monitor counters: Bytes Sent to Replica/sec, Log Bytes Flushed/sec, Redone Bytes/sec, etc. (Microsoft Learn)

22. **How to monitor AG synchronization health?**
    Using AG dashboard in SSMS; checking replica states (SYNCHRONIZING, SYNCHRONIZED, NOT SYNCHRONIZING), checking send/redo queue backlog, latency.
23. **How to detect when secondaries are falling behind?**
    Look at redo_queue_size or log_send_queue_size; also metrics for log bytes flushed vs log bytes sent/received.
24. **What are typical thresholds / alert conditions for lag / queue sizes?**
    It depends on SLA, but for example, if redo lag is more than X seconds, or if log send queue size exceeds some MB threshold, or RTO / RPO approaching SLA, generate alerts.
25. **How to monitor failover time?**
    Measure both AG-level (listener failover & cluster resource movement) and DB-level (databases becoming available/readable on new primary).
26. **How to monitor the network bandwidth used by AG replication?**
    Perfmon counters on throughput; also watch for peaks, small bursts ("spikes") as logs flush; consider compression if used.
27. **How to monitor resource usage on the secondary (CPU / I/O / memory)?**
    Because redo on secondary uses CPU and disk, and if secondary is also serving read workloads, resource contention is possible. Monitor those resources to ensure secondary is not overloaded.

28. **How to track performance difference between primary and secondary for particular queries?**
    Compare execution plans, statistics, wait times; sometimes read-only on secondary may run queries slower if statistics / maintenance are lagging.
29. **What metrics indicate problems in AG performance?**

- Large log send queue
- High redo latency
- Secondary lag behind for long periods
- Elevated IO latency on primary or secondary disks
- Increased transaction commit times

30. **How to monitor cross subnet / multi-site AG performance?**
    Include network latency, packet loss, throughput capacity; make sure WSFC cluster health options are configured properly (e.g. health detection, resource DLL settings).

31–40: Troubleshooting & Edge Scenarios

31. **What are typical causes when AG commit latency is high?**
    Disk latency (log disk on primary or harden disk on secondary), network latency, slow redo on secondary, insufficient CPU, or overburdened system.
32. **Why is AG failover taking too long?**
    Many databases in AG, slow database recovery, large log backlog, slow network, mismatched hardware, cluster quorum issues.
33. **Why is there data loss despite synchronous mode?**
    Misconfiguration (e.g. not fully hardened on secondary), network disconnects, or partial failures; secondary not healthy or lagging.
34. **What happens if secondary is read-only and long-running queries block redo?**
    Read-only workloads on secondary can compete for resources. While they don't block redo per se, contention on I/O or CPU can slow redo.
35. **Can maintenance operations (index rebuild, backups) on secondary affect AG performance?**
    Yes. If backups run on secondary or index rebuilds, they consume I/O and CPU that could slow redo or affect send/receive throughput.
36. **If primary is under heavy load, can AG replicas exacerbate the issue?**
    Yes. The more secondaries, or if secondary replicas are writable/readable and doing work, the primary must send logs to each, increasing network, CPU, I/O load.
37. **What are common misconfigurations that degrade performance?**

- Asynchronous commit used where sync is expected
- Network misconfigured (single NIC, virtualized, oversubscribed)
- Slow storage on secondary
- Unbalanced hardware between replicas

38. **Why might there be differences in execution plan between replicas?**
    Differences in statistics, missing indexes, hardware (CPU / I/O differences), read-only workloads, hardware configuration, parameter sniffing.
39. **Why is automatic page repair / corruption slower in AG environment?**
    Because the secondary has to sync and apply changes, the overhead of data movement plus recovery can slow things; also the more replicas, the more complexity for consistency checking.
40. **What issues arise with multi-subnet AGs?**
    Multi-subnet introduces complexities with listener latency, DNS timeouts, failover detection, possibly asymmetric network performance. Requires proper "MultiSubnetFailover=True" in connection strings, and validating network paths.

41–50: Advanced & Scaling Scenarios

41. **What is the maximum number of replicas supported, and what performance trade-offs are there?**
    SQL Server supports up to 8 replicas (depending on version). More replicas means more overhead on primary to send log data, greater chance of lag, more network and I/O load.
42. **How do large numbers of AG databases affect resource usage?**
    As seen in scalability experiments, when AG has many DBs (hundreds), failover time may become large, resource consumption (redo, send, CPU) spikes. (Microsoft Tech Community)
43. **How does SQL Server version matter for AG performance?**
    Later versions (SQL Server 2016+) include improvements: more efficient log block transfer, improved redo, parallel hardening etc. E.g., 2016 "turbocharged" improvements. (Microsoft Tech Community)
44. **What are the performance considerations for asynchronous replicas in distant DR sites?**
    Large log backlog, potential data loss in failover, network outages, latency spikes. Need to size network appropriately and monitor carefully.
45. **How to balance read workloads across readable secondaries?**
    Use routing lists, read-only workloads, you may need to design load balancing, use listener with read-intent routing, or have applications direct read workload to secondaries.
46. **What about backup on secondary replicas—pros/cons?**
    Pros: offloads backups from primary; cons: backups consume I/O / CPU / storage on secondary, which can reduce redo performance or interfere with read workloads.
47. **How to do performance testing for Always On before production deployment?**
    Simulate workloads (read/write), measure log send / redo under load, test failover times, network latency, disk I/O latency, monitor metrics, test under realistic peak loads.
48. **What are best practices for automatic seeding / initial synchronization?**
    Use fast network; consider using backup/restore manually if seeding over slow or high-latency links; monitor seeding; possibly break into subsets; avoid doing seeding during peak load.
49. **What is the impact of encryption (e.g., Transport Layer Security, TDE) on AG performance?**
    Encryption adds CPU overhead on primary and secondary (especially during transfer and hardening). Newer hardware helps. Plan accordingly.
50. **How to scale Always On to geo-distributed / multi-datacenter deployment?**
    Use asynchronous replicas, possibly cascading replicas, network optimization, consider disaster recovery site, ensure data redundancy, optimize network paths, tune commit modes, ensure that listeners and DNS are properly configured for multi-site topologies.

🧠 **Replication & Performance Scenarios FAQs — SQL Server DBA**

1-10: Basics & Types of Replication

1. **What are the different types of SQL Server replication?**
   - Snapshot replication: point-in-time copy of data/articles. (CData Software)
   - Transactional replication: continuous change capture & apply committed transactions. (CData Software)
   - Merge replication: bi-directional updates, conflict detection/resolution, offline/connected work. (Documentation Help)
   - Peer-to-peer transactional replication: as a special form of transactional, multiple nodes can both publish/subscriber. (devart.com)
2. **When should I use snapshot replication vs transactional vs merge?**
   - Snapshot when data changes infrequently or latency tolerable, simpler topology.
   - Transactional when you need near-real time changes, read scale-out, reporting, low latency.

- Merge when subscribers may be disconnected or for multi-site, bi-directional updates.

3. **What are the key components/agents involved in replication?**
   - Publisher, Distributor, Subscriber, Publication, Articles, Subscriptions. (devart.com)
   - Agents: Log Reader Agent (for transactional), Distribution Agent, Snapshot Agent, Merge Agent. (CData Software)
4. **What is latency in replication? What causes it?**
   Latency = delay from when data change commits on Publisher to when it is propagated/applied on Subscriber. Causes include large transactions, slow network, overloaded distributor, slow disk I/O, contention in agents.
5. **What is filter / row / column filtering in replication, and how does it impact performance?**
   You can limit what data (rows or columns) gets published or subscribed. Filtering reduces data volume, but adds overhead during snapshot generation and metadata/tracking. If filters are complex or many, can degrade merge/snapshot performance.
6. **What is conflict resolution in merge replication?**
   Since changes can occur on both Publisher and Subscriber, conflicts happen; rules (priority, custom resolver) decide which change wins. Conflict detection and resolution introduces overhead.
7. **How is schema change managed in replication?**
   Schema changes must be replicated appropriately. Some replication types support limited schema changes; altering tables with many columns, or doing multiple ALTERs sequentially vs single statement matters. Merge replication especially is sensitive. (sqlshack.com)
8. **How does transactional replication ensure ordering and transactional consistency?**
   Uses transaction log reading and ensures order of committed transactions; the Log Reader picks up committed transactions in order, and Distribution Agent applies them in same boundaries.
9. **Can replication affect transaction log behaviour?**
   Yes. With transactional/merge replication, the log cannot be truncated until transactions that are to be replicated are processed. If replication is slow, log reuse wait may stay in 'REPLICATION' state, meaning log grows.
10. **What are distributor best practices?**
    - Dedicated server if load is high.
    - Good disk I/O on distributor database.
    - If possible, remote distributor rather than same as publisher to reduce load.
    - Proper sizing of tempdb/distribution DB.

11-20: Performance Tuning & Bottleneck Scenarios

11. **How to tune transactional replication for high throughput?**
    Some tactics include:
    - Batch sizes (CommitBatchSize) for Distribution Agent, ReadBatchSize for Log Reader Agent. (Microsoft Learn)
    - Reduce transaction size (many small transactions vs very large ones). (Microsoft Learn)
    - Use a remote distributor to offload work. (Microsoft Learn)
12. **What parameters of agents affect performance?**
    - Log Reader Agent: ReadBatchSize, polling interval etc.
    - Distribution Agent: CommitBatchSize, polling, number of threads, etc.
    - Snapshot Agent: snapshot folder, network/memory/disk during snapshot creation.
13. **How do large transactions affect replication?**
    They can cause long latency, large log buildup, slower agent processing, possible timeouts. Better to break into manageable batches.
14. **What are I/O bottlenecks in replication?**
    - On the publisher: transaction log write and log read (for transactional).
    - On distributor: distribution DB I/O.
    - On subscriber: apply operations I/O.

15. **How does network capacity/latency affect replication?**
    Slow or unreliable links cause high latency. Packet loss, retries degrade throughput. Compression if available, or scheduling off-peak help.
16. **What performance impact comes from using filters, especially parameterized row filters?**
    These can add complexity to snapshot enumeration, filtering overhead, and increase metadata/tracking work. Might slow merge / snapshot syncs.
17. **What about read/write skew in peer-to-peer transactional replication?**
    Since each node is both publisher and subscriber, conflicting writes or high write activity can cause latency or conflicts; careful schema design and conflict detection (if any) needed.
18. **How to tune merge replication performance?**
    - Limit number of conflicts.
    - Use efficient filtering.
    - Avoid lots of triggers/overhead.
    - Ensure system tables used for merge tracking are performant.
    - Regular cleanup of metadata.
19. **How to schedule snapshot replication for large publications?**
    - Off-peak hours.
    - Possibly partitioning snapshot articles (if possible).
    - Use compression of snapshot files if supported.
20. **What happens to performance if agents fall behind?**
    Distribution agent backlog increases; transactional latency spikes; transaction log growth; subscribers stale; possible resource contention catching up.

21-30: Monitoring & Diagnostic Scenarios

21. **What metrics / DMVs to monitor for replication performance?**
    - sql server: replication monitor tools to view status.
    - DMVs: msdb replication related tables, distribution database stats.
    - Log Reader Agent latency, Distribution Agent latency.
    - System performance counters: disk I/O, network throughput, CPU, memory.
22. **How to detect replication latency / lag?**
    - Replication Monitor (Publisher, Distributor, Subscriber).
    - Monitor difference in LSNs between publisher & subscriber.
    - Observe log send / distribution queue size.
23. **How to detect a bottleneck in the Distributor?**
    If the distribution database has high latency, slow writes, or the distribution agent is frequently throttled; if CPU, I/O waits high on that server.
24. **What to watch for in transaction log reuse waits?**
    If log_reuse_wait_desc = 'REPLICATION' means replication is holding up log truncation. This may cause huge log growth.
25. **How to capture replication errors / agent failures?**
    Use SQL Agent jobs logs, Replication Monitor, and error tables (MSrepl_errors etc.). Also monitor for timeouts, agent restarts.
26. **How to monitor schema changes propagation?**
    Keep track in agent logs when schema changes happen; ensure subscribers schema matches expected.
27. **How to check for undistributed commands or unsynchronized schema/articles?**
    Use Replication Monitor; use stored procedures like sp_helparticle, sp_helppublication, etc., and query metadata.
28. **How to measure distribution agent performance over time?**
    Log and chart throughput (rows/sec, commands/sec), latency, CPU/disk usage.
29. **How to detect merge replication conflict hotspots?**
    Monitor conflict tables; check which articles/tables have most conflicts; maybe conflict resolution rules are inefficient.

30. **What kinds of alerts are helpful in replication environments?**
    Alerts when latency exceeds a threshold, job failures, log growth, unsent commands, subscriber unreachable, data integrity discrepancies.

31-40: Edge & Real-World Scenarios

31. **What performance issues arise when reinitializing a subscriber?**
    Snapshot application can be heavy; large data movement; locking; possibly blocking on subscriber; time to apply snapshot can be long.
32. **What if distribution database becomes very large?**
    Old history and undistributed commands accumulate; disk usage grows; backups take longer; agent performance degrades. Need cleanup & pruning.
33. **How does replication interact with large deletes / updates / bulk inserts on publisher?**
    These generate large amounts of data to send; can cause log growth; agent may lag; subscriber may get overwhelmed applying changes; schedule appropriately or break into batches.
34. **What performance cost comes from join / computed / indexed columns in articles?**
    More complex articles (with computed, indexed, triggers) cost more to replicate; trigger overhead; increased metadata/tracking.
35. **Can subscriber apply latency be higher than distribution send latency?**
    Yes. Even if data arrives quickly, applying it (especially merge) may take time due to conflicts, schema issues, indexing, locking.
36. **What issues appear when subscribers are on slower hardware / disks?**
    Slower apply, lag, possible blocking, backlog in agents, degraded consistency.
37. **How does system load (other workloads) on publisher / distributor / subscriber affect replication?**
    Resource competition (CPU, I/O, network) degrades replication performance; agents contend; could throttle; need resource balancing.
38. **What happens in case of network blips or disconnects?**
    Transaction loss is avoided (replication is durable), but latency spikes, agent retries, possible timeouts, bigger backlog; could lead to out-of-sync states.
39. **What are performance differences in replication across WAN vs LAN?**
    Across WAN latency, bandwidth constraints, higher error potential; snapshot and transactional replication performance worse across WAN; must tune accordingly.
40. **How to handle schema or article changes when replication is busy or under load?**
    Plan offline windows; ensure minimal impact; use single ALTER statements where possible; avoid altering many tables at once; test carefully.

41-50: Advanced / Scaling / Best Practices

41. **Is peer-to-peer replication viable for scale-out / geo-distributed writes?**
    It can be but requires careful design: conflict avoidance (e.g. ensure each node writes distinct primary keys), topology, latency between nodes, conflict resolution, schema consistency.
42. **How to scale large transactional replication topologies (many subscribers, many articles)?**
    - Use remote distributors, possibly multiple distribution databases.
    - Limit number of articles per publication if articles are very large.
    - Use filtering.
    - Use multiple agents where possible.
43. **How to test replication performance before going to production?**
    Create a similar topology; replay production-like workload; simulate network latency; measure agent and segment throughput; monitor log growth and latency.

44. **What are trade-offs between replication vs Always On / AG / log shipping / mirroring for read scale-out or HA/DR?**
    Replication gives flexible read scale-out, selective data, multi-site; but complexity, possible latency, conflict in merge; AGs/AlwaysOn may give better HA, but less flexibility in article selection; log shipping simpler but more lag, not real-time.
45. **What best practices minimize replication overhead?**
    - Only replicate what is necessary (filtering, article selection)
    - Keep subscriber schema lean, with proper indexing.
    - Use appropriate hardware/disks for log, distributor, agents.
    - Monitor and prune distribution DB and replication metadata.
46. **How to manage replication during maintenance windows / backups / index rebuilds?**
    Avoid snapshot during busy periods; schedule index rebuilds on subscriber when possible; backups on subscriber if allowed; ensure agents paused if major changes; inform replication of schema changes carefully.
47. **What security / permissions considerations affect performance?**
    Agents need proper permissions; network encryption overhead (if using SSL/TLS); ensure that LAN security (firewalls etc.) isn't introducing latency; minimal overhead replication operations.
48. **What happens when the distribution database is on slower I/O / shared storage?**
    It becomes a bottleneck. High write/read demand slows down log reader / distribution agents; may degrade latency and throughput.
49. **How to handle version upgrades / compatibility when using replication?**
    Be aware of deprecated replication features; test replication after upgrade; ensure that replication agents are updated; check schema change behavior; test cross-version compatibility.
50. **When to consider alternatives to built-in replication for very high-Performance or global workloads?**
    For very low latency, large scale, high global distribution: consider data-warehouse / ETL-based replication, Change Data Capture + custom sync pipelines, or third-party / third-party tools, or use Always On read replicas / geo-replication services, depending on SQL Server / cloud offerings.

⚠ TempDB & Contention Scenarios — Top 50 FAQs for SQL Server DBAs

1–10: Fundamentals & What is Contention in TempDB

1. **What is tempdb?**
   A system database used for all sorts of temporary or intermediate storage: temp tables, table variables (when spilling), work tables, version store, sorts/spools, etc. It is recreated on each SQL Server restart.
2. **What kinds of contention occur in tempdb?**
   - **Allocation contention** on PFS/GAM/SGAM pages. (Microsoft Learn)
   - **Metadata contention** (catalog/sysobj, temp table cache, etc.). (support.microsoft.com)
   - **Latch waits** due to many concurrent allocations / deallocations. (Microsoft Learn)
3. **What are PFS, GAM, SGAM pages?**
   System pages used to track allocation of pages/extents in a database:
   - **PFS** (Page Free Space) tells which pages are free, used, mixed, etc.
   - **GAM** (Global Allocation Map) tracks which extents are allocated wholly.
   - **SGAM** (Shared Global Allocation Map) tracks mixed extents (some allocated, some free).
4. **What symptoms suggest tempdb contention?**
   - High waits of type PAGELATCH_… (e.g. PAGELATCH_EX, PAGELATCH_UP, PAGELATCH_SH) on resources in tempdb. (Microsoft Learn)
   - Processes blocking each other waiting on allocation pages. (Microsoft Learn)
   - CPU underutilization: although many tasks are waiting, CPU may appear idle. (Microsoft)
   - Slow query performance, particularly when using temp objects, large sorts, hash operations.

5. **How does version (SQL Server version) affect tempdb contention?**
   Later versions (esp. SQL Server 2022) include improvements in tempdb allocation concurrency (e.g. allowing concurrent updates to GAM/SGAM pages, improvements in how PFS pages are used) thereby reducing contention. (Microsoft)
6. **What workloads tend to stress tempdb most?**
   - Many concurrent sessions creating/dropping temp tables. (Microsoft Learn)
   - Large sorts, hash joins/aggregations, spill to disk. (Microsoft)
   - Snapshot isolation / read committed snapshot / versioning scenarios. (Microsoft)
   - Online index operations, DBCC CHECK, work tables, large object (LOB) temporary operations.
7. **What is temp table caching and how does it relate to contention?**
   Temp table caching allows reuse of temp tables created within stored procedures (if they match certain criteria), so metadata operations (e.g. create/drop) may be avoided; this reduces metadata contention. But not everything is eligible, and certain operations (ALTER TABLE, indexes, changing schema) break caching. (Microsoft)
8. **Why are all tempdb data files recommended to be the same size & growth settings?**
   To ensure proportional fill (SQL Server's mechanism for using files evenly). If sizes/growth differ, one file may fill or be used more, causing contention still. (docs.aws.amazon.com)
9. **How many data files should tempdb have?**
   General guidance:
   - If ≤ 8 logical processors: use the same number of tempdb data files as logical processors.
   - If >8 processors: start with 8 files, then increase in increments of 4 if contention still occurs, up to number of logical CPUs. (docs.aws.amazon.com)
10. **Should tempdb data files be placed on dedicated storage?**
    Yes. Fast I/O (low latency / high throughput) for both data and log files is important. It helps with spills, sorting, version store, etc.

11–20: Diagnostics & Monitoring

11. **What DMVs or system views help identify tempdb contention?**
    - sys.dm_os_waiting_tasks (to see what waits, and check resource_description for “2:1:1”, “2:1:2”, “2:1:3” etc) for PFS/GAM/SGAM pages. (Microsoft Learn)
    - sys.dm_exec_requests / sys.dm_exec_sessions to see who is waiting or blocked.
    - sys.dm_os_memory_cache_counters to see temp table cache usage. (support.microsoft.com)
12. **How do you detect PFS/GAM/SGAM contention?**
    By finding waits of type PAGELATCH_EX, PAGELATCH_SH, PAGELATCH_UP where the resource_description points to PFS/GAM/SGAM pages in tempdb (e.g. “2:1:1” etc). (Microsoft Learn)
13. **What scripts/tools help find allocation contention?**
    - Scripts that query sys.dm_os_waiting_tasks filtering for PAGELATCH waits on “2:x:1”, “2:x:2”, etc. (SQLServerCentral)
    - Monitoring tools and custom alerts for tempdb pagelatch waits.
14. **How to monitor tempdb usage/trends?**
    - Track number of tempdb data files usage, tempdb space usage (data and log).
    - Monitor spills to tempdb (hash joins, sorts).
    - Version store size if using snapshot isolation.
    - Monitor wait stats over time (esp PAGELATCH waits).
15. **What wait types to watch for?**
    - PAGELATCH_EX, PAGELATCH_SH, PAGELATCH_UP on tempdb pages (especially PFS/GAM/SGAM). (Microsoft Learn)
    - CMEMTHREAD, SOS_CACHESTORE spinlock waits for temp table metadata cache contention (depending on version) (Microsoft)
16. **How to measure tempdb metadata contention (sysobjvalues etc.)?**
    Use wait stats, memory-cache counters, and check CACHESTORE_TEMPTABLES in sys.dm_os_memory_cache_counters. If

entries are high and many sessions are creating/dropping temp tables (especially with schema changes), you may see contention. (support.microsoft.com)

17. **How to find tempdb "who is using space" / large usage queries?**
    Use DMVs like sys.dm_db_task_space_usage and sys.dm_db_session_space_usage. Also extended events or traces capturing large tempdb allocations.
18. **How to capture spills to tempdb?**
    Execution plan include operators that spill (hash/merge/spool). Query plan warning messages. Also monitor sys.dm_exec_query_stats / sys.dm_exec_requests to find plans that report spill behavior.
19. **How to detect skew in tempdb usage across files?**
    Check sys.dm_io_virtual_file_stats (if files are on separate file sets), or check page allocations / wait /iou per file; examine wait/resource descriptions pointing to specific file IDs.
20. **What impact do auto-growth and file initial size have on contention?**
    Frequent auto-growth events can cause pauses; if files are too small, they'll fill, cause allocation waits. Large initial size helps reduce growth events. Growth settings must be the same for all tempdb data files to avoid imbalance.

21–30: Configuration & Best Practices to Alleviate Contention

21. **How many tempdb data files should I configure?**
    As above: match logical CPUs (≤8), start at 8 for larger, then increment in 4s until contention is acceptable.
22. **Should tempdb data files be equal size?**
    Yes. To avoid one file being hot while others are underutilized. Equal size + equal growth.
23. **Use of trace flags like 1118, 1117, etc.**
    - TF 1118 (for older versions) to force uniform extent allocations, reduces SGAM contention. (Microsoft Learn)
    - TF 1117 (older) affects auto growth behavior.
24. **Place tempdb data/log on fast storage**
    Use SSD / NVMe; minimize latency, avoid shared or slow disks; separate log and data if possible.
25. **Configure auto-growth appropriately**
    All files should have similar auto-growth settings (size and increment). Avoid tiny growth; prefer fixed MB rather than % growth.
26. **Pre-size tempdb files large enough**
    Avoid frequent growth during peak. Set initial sizes based on expected usage.
27. **Reduce unnecessary tempdb usage in queries**
    - Avoid unnecessary ORDER BY, GROUP BY if not needed.
    - Avoid overusing temp tables / large table variables.
    - Ensure queries are efficient so they don't spill to tempdb.
28. **Optimize versioning usage**
    If using snapshot isolation / RCSI, monitor and configure version store cleanup (retention), ensure sufficient disk space, monitor version store size.
29. **Upgrade patch / cumulative updates**
    Some tempdb contention issues have been fixed in CUs (e.g. heavy tempdb contention in SQL Server 2016/2017 fixed in specific CUs). (support.microsoft.com)
30. **Use latest SQL Server version enhancements**
    SQL Server 2022 has enhancements for GAM/SGAM concurrency, improved metadata cache handling. Using newer version helps reduce legacy contention. (Microsoft)

31–40: Real-World Scenarios & Edge Cases

31. **What happens when tempdb is filled up / runs out of space?**
    Queries that need tempdb fail (errors around workspace or tempdb allocation). Can cause service disruption.

32. **How to manage tempdb size growth retention?**
    Cleanup of objects, version store, ensure sessions finish; avoid open transactions holding version store; restart clears tempdb (but impacts uptime).
33. **Impact of tempdb metadata cache invalidation**
    When temp tables are altered or dropped or schema changes, metadata cache for temp tables invalidates, causing more overhead in subsequent temp table creation. (support.microsoft.com)
34. **Large number of sessions creating/dropping temp tables & indexes**
    Causes spikes of allocation contention & metadata contention. Possibly change design (reuse temp tables in procedures, avoid frequent schema changes).
35. **Online index operations using tempdb**
    The "shadow" operations use tempdb for sorts and versioning, causing tempdb activity.
36. **Spill operations (sort, hash) causing tempdb IO bottleneck**
    If memory grants insufficient, operators spill to tempdb, causing IO stress.
37. **Snapshot isolation / RCSI placing burden on tempdb version store**
    Version store can grow if transactions are long-running or don't complete, blocking cleanup.
38. **DBCC CHECKDB's impact on tempdb**
    It uses tempdb for internal work, especially for sorting / checking temp structures. On large databases, this can stress tempdb.
39. **When many small tempdb allocations/deallocations occur (e.g. temp tables in loops)**
    Repetitive create/drop is worse than reuse; may consider pooling, reuse, or temp tables created once, truncated, reused.
40. **Interference between user tempdb usage and internal SQL Server operations**
    Internal operations (e.g. spool, sort, online index, metadata operations) compete with user tempdb usage for IO, latches, etc.

41–50: Troubleshooting, Version Differences, & Advanced Tuning

41. **How to quickly test if tempdb contention is the issue?**
    - Check for waits on PFS/GAM/SGAM.
    - Reduce number of tempdb data files temporarily to see effect.
    - Create load test with many temp tables / concurrent sessions.
42. **What version-specific fixes exist?**
    E.g. SQL Server 2016/2017 had a known issue: heavy tempdb contention when many temp tables created and schema changes invalidate the cache; fixed in certain cumulative updates. (support.microsoft.com)
43. **How SQL Server 2022 improves tempdb contention**
    - Concurrent GAM/SGAM updates.
    - Better PFS page round robin allocation. (Microsoft)
44. **When to use trace flags / startup options**
    For older versions, TF 1118; in some circumstances TF 1117; but must test for side effects (e.g. wasted space).
45. **Impact of mixed vs uniform extents**
    Uniform extents reduce SGAM contention for allocations. Mixed extents (when objects are small) can lead to more contention.
46. **When adding more data files stops helping**
    There is a point of diminishing returns. Adding too many files increases management overhead; balance with file count vs logical CPUs vs workload.
47. **Best layout for tempdb in VMs / hypervisors**
    - Use local SSD or ephemeral storage (if durable enough).
    - Avoid shared network or slow disk.
    - Ensure IO path is optimized.
48. **Monitoring spill operators**
    - Capture missing memory grant waits.

- Use Query Store / Extended Events to find queries with sort/hash spills.

49. **How to manage version store and cleanup bottlenecks**
    - Identify long-running transactions preventing cleanup.
    - Monitor version_store_cleanup waits (if present).
    - Ensure sufficient space and I/O for version store.
50. **Plan for maintenance / worst-case events**
    - Pre-size tempdb for peak usage (e.g. during large report runs or batch jobs).
    - Ensure failover nodes have matching tempdb configuration.
    - Document and test what happens under tempdb exhaustion.

⚙ **Backup & Restore Performance Scenarios FAQs — SQL Server DBA**

1-10: Fundamental Concepts & Trade-Offs

1. **What backup types are there, and how do they affect restore times?**
    - *Full backups* take the longest but provide baseline.
    - *Differential backups* are faster to create & restore since they include only changes since last full backup.
    - *Transaction log backups* used for point-in-time recovery; restore involves replaying logs.
    - Bulk-logged vs full recovery models: log backups under bulk-logged can reduce log size but may affect what you can restore to.
2. **What is the trade-off between backup compression vs speed & resource usage?**
   Compression reduces disk space and can decrease I/O from and to storage, but uses extra CPU. If CPU is saturated, backups (or restore) under compression may slow down other workloads. Also, restoring compressed backups involves decompression overhead.
3. **How does hardware (storage / disks / I/O subsystem) affect backup/restore performance?**
   Slow disks, high latency, bottlenecked I/O, or network storage (NAS, SMB, etc.) can significantly degrade both backup and restore. Using fast local SSD / NVMe or premium storage helps.
4. **What is the impact of backup file layout (# of files, striping) on restore performance?**
   Using multiple backup files (striped backups) across multiple disks allows parallel I/O during both backup and restore. This often speeds up the operations. (Henk van der Valk)
5. **How do MAXTRANSFERSIZE and BUFFERCOUNT parameters affect restore throughput?**
   Raising MAXTRANSFERSIZE (size of each I/O transfer) and increasing BUFFERCOUNT (number of buffers used during restore) can help increase throughput. But too many buffers can lead to memory pressure. (SQLServerCentral)
6. **What is Instant File Initialization (IFI), and when does it help?**
   IFI allows data files (MDF, NDF) to skip zero-filling when growing or restoring, which can greatly speed up restores that involve moving or growing files. It doesn't apply to log files, which must still be zeroed. Useful especially during restores with WITH MOVE. (Database Administrators Stack Exchange)
7. **What about sector size / disk alignment considerations?**
   Restoring to disks with 4K sector size under certain compressed backup scenarios is documented to be slower in some SQL Server versions. Microsoft has released fixes for particular cases. (Microsoft Support)
8. **How does version / edition of SQL Server matter?**
   Newer versions often have optimizations (e.g. for compressed backups, parallelism, redo improvements), bug fixes, better readahead behaviors. Also some features (e.g. backup compression) are not available in all editions. Having the latest cumulative updates / service packs can solve unexpected slowdowns.
9. **What is the effect of backup destination / target (disk, network share, Azure blob, etc.)?**
    - Writing to local fast disks is best.
    - Network shares or over SMB can introduce latency and bandwidth constraints.
    - Cloud storage or remote storage introduces more latency and possibly throttling.

- Backup to URL (in Azure) or blob storage has its own trade-offs (bandwidth, transfer times, security overhead).

10. **How frequently should backups and differential/log backups be scheduled to balance restore speed vs resource usage?**
Depending on DB size, change rate, SLA (RPO / RTO). For high-transaction systems, frequent log backups (e.g. every 5-15 minutes), daily full backups, and regular differentials help limit log redo during restore and reduce downtime. Overdoing backups can add overhead though.

11-20: Common Performance Bottlenecks & Problems

11. **Why are backups causing application slowdown?**
Could be because backup operations consume I/O or disk throughput, compete for disk, saturate network, consume CPU (especially with compression), or impact disk cache / read/write paths. Also if backup uses same storage as your data files or log files.
12. **Why is a restore operation slow even though backup is fast?**
    - Restore includes file initialization, moving data to new location, zeroing out files (unless IFI is enabled), redo of transaction log, and possibly decompression.
    - Restoring to different hardware / slower storage.
    - Memory / buffer settings (MaxTransferSize / BufferCount) not optimized.
13. **What happens when restores use WITH MOVE?**
If you move data/log files during restore, the new files may need initialization (zeroing) and possibly growth operations which slow down the overall restore. IFI helps for data files. Also moving may cause the system to allocate and create file structures which takes time.
14. **Issues with restoring encrypted or TDE-enabled databases**
Encryption adds overhead: backup must include encryption metadata/keys; restore must decrypt, which can use CPU; storage of encrypted backup maybe different. Also, moving between servers needs proper key management.
15. **Impact of Change Data Capture (CDC), Change Tracking, etc., on backup/restore**
Some versions had known issues: for example, large number of CDC tables could cause slow restore due to internal metadata operations. There are hotfixes to address them. (Microsoft Support)
16. **Slow restore because of bad readahead behavior**
Restore uses readahead, but if MAXTRANSFERSIZE is small, or buffer settings are low, or disks are not allowing large sequential reads, then the I/O subsystem may limit throughput.
17. **Restore of compressed backup to disk with specific sector sizes**
As noted, disk sector alignment or characteristics (e.g. 4K sector disks) can degrade performance for compressed backups until specific fixes or patches are applied. (Microsoft Support)
18. **Memory pressure during restore or backup**
Having too many buffers (buffercount), compression, or decompression uses memory. If server is already under load, adding backup/restore operations can cause paging or memory contention, which slows everything.
19. **Log backups taking long / blocking restore windows**
For very busy databases, log backup size grows; if they are large and being sent over network, or storage is slow, then the log backup operation can take long. Also, restoring many log backups (for point in time) takes time.
20. **Network latency / bandwidth limitations**
Especially in distributed or cloud environments. If backup destination or source (for restore) is across network, throughput is limited. Reads/writes over network may need retries, be subject to throttling.

21-30: Tuning & Best Practices

21. **Use striped backups (multiple files) for both backup and restore**
As noted, backup to multiple files across different disks speeds up both backup and restores. (Henk van der Valk)
22. **Optimize buffer settings: BufferCount, MaxTransferSize**
Testing to find sweet spot of buffer size and count to feed your I/O subsystem optimally. Use large transfer sizes if disks support, but don't exceed what memory / I/O can handle. (SQLServerCentral)

23. **Enable Instant File Initialization where possible**
    For restoring/moving data files. Makes big differences especially for large databases.
24. **Pre-size backup files / ensure enough backup disk space**
    Avoid auto-growth of backup storage or backups failing because disk is full; also large files may get fragmented.
25. **Use compression or backup software that supports compression**
    Where CPU allows. Compressed backups can reduce both I/O and storage needed.
26. **Perform restores regularly to test both correctness & performance**
    Theoretical restore performance is not enough; must practice restores so you understand how long under your hardware and with your data.
27. **Use differential backups strategically**
    Full nightly, differential daily or mid-day, logs more frequently. This reduces the volume of recovery (fewer logs to apply).
28. **Use fast storage for backup target**
    Use disks with good sequential write performance; avoid slow disks; use RAID10 rather than RAID5 for backup targets (since parity overhead).
29. **Offload backups to secondary replicas / read-only nodes**
    If using Availability Groups or replication, offload backups to secondaries so primary is not taxed.
30. **Avoid contention of backups running simultaneously**
    If multiple backups run at same time, or backup + restore + other heavy I/O on the same disks, they compete. Stagger or separate.

31-40: Monitoring & Metrics to Watch

31. **Backup throughput (MB/sec) and I/O utilization**
    Monitor how many MB/sec you get during backup; check disk subsystem latency (read/write), throughput, queue lengths.
32. **CPU usage during backup / compression**
    Compression uses CPU; monitor CPU usage, especially if backup coincides with peak workloads.
33. **Wait stats during restore (e.g. LOGBUFFER, IO waits)**
    E.g. LOGBUFFER waits when log redo is slow; IO waits when reading/writing backup file or writing data pages.
34. **Time taken for restore vs. target RTO**
    Track actual restore times—full, differential + logs—to see if they satisfy your SLA.
35. **Backup and restore blocking / impact on concurrent workloads**
    Monitor lock waits, I/O waits, tempdb usage, etc., when backup/restore runs.
36. **Size of transaction log backups & chain of log backups**
    If log backups are too big, application of them will be slow; also affects restore time.
37. **Space usage & free space on backup storage**
    Running out of space can cause failures, or slowdowns with fragmented storage.
38. **Number of backup files / file handles open**
    Too many parallel backup files can create overhead; balance with what disks can handle.
39. **Disk sector alignment and physical block sizes**
    Mismatch between disk block sizes, file system cluster sizes, or underlying storage (especially in virtual environments) can degrade throughput.
40. **Effects of encryption/TDE / backup to URL or cloud storage**
    Backup encryption or using .BAK backups over cloud or blob storage introduces latencies; monitor end-to-end times including network.

41-50: Edge Cases, Version Issues, Troubleshooting & Scenarios

41. **Restore failures due to corrupted backups / media errors**
    Use CHECKSUM option during backup; verify using RESTORE VERIFYONLY; maintain multiple backup copies; use reliable storage. (Microsoft Learn)

42. **Slow restore in presence of CDC / large number of tables with metadata**
    As noted, in older versions, enabling CDC on large DBs with many tables caused slow restores; hotfixes are available. (Microsoft Support)
43. **Restoring via WITH MOVE taking unusually long**
    Because of file initialization, moving to different storage, slower disk or file fragmentation; enabling IFI and using fast storage helps.
44. **Restore of large compressed backups / backups with many files**
    Useful but trade-offs: managing many files complicates management; file/disk failures more impactful; need to ensure good I/O balance across files.
45. **Restoring between different SQL Server versions / compatibility issues**
    Backups cannot be restored to earlier versions; features differences may cause performance differences.
46. **Effect of backup sector size / disk block size mismatches**
    Mismatch or suboptimal alignment or small I/O sizes can hurt performance; some known slowdowns when using compressed backups on 4K sector disks. (Microsoft Support)
47. **Dealing with very large databases (VLDBs)**
    For very large DBs (hundreds of terabytes), consider file/filegroup backups, partial backups, split backups, parallelism, infrastructure that supports large sequential I/O.
48. **Handling backup and restore in cloud / hybrid scenarios**
    Consider network, bandwidth, egress costs, storing backups in object storage or snapshots; sometimes snapshot / incremental / differential backups to cloud reduce time.
49. **Scheduling consideration (off-peak / maintenance windows)**
    If possible, schedule large full backups and large restores during off-peak times to reduce interference; differential or log backups can be more frequent and lighter weight.
50. **Documenting / testing recovery procedures & disaster recovery drills**
    It's not enough to build backup schedules: you must test restores, measure how long they take, simulate worst case, and verify that backups are usable, integrity is good, and that you meet RTO/RPO under real conditions.